DOCUMENTO DE ARQUITECTURA DE SOFTWARE (SAD)

SISTEMA MULTIAGENTE BASADO EN MICROSERVICIOS Y CONTENEDORES PARA LA AUTOMATIZACIÓN DE LA PRODUCCIÓN Y PUBLICACIÓN DE CONTENIDO DIGITAL EN LA EMPRESA ITY SAS

TRABAJO DE GRADO -- PRODUCTO TÉCNICO FORMAL **Cumplimiento de los Objetivos Específicos OE2 y OE3 • Alineación Integral con la SRS IEEE Std 830-1998**



Autor: Alvaro Felipe Lopez Russi • Código: 67000867 • `aflopez67@ucatolica.edu.co` **Universidad Católica de Colombia • Facultad de Ingeniería** Programa de Ingeniería de Sistemas y Computación • Bogotá D.C., Octubre 2026 • Versión 3.0 Oficial

CONTROL DOCUMENTAL Y METADATOS DEL DOCUMENTO DE ARQUITECTURA

Atributo de Gobernanza	Especificación Técnica Formal
Identificador del Documento	`ITY-MAS-SAD-001` (Software Architecture Document - SAD)
Plantilla y Marco Metodológico	**arc42 (Versión 7.0 ES) • ISO/IEC/IEEE 42010:2022** (*Architecture Description*)
Estándar de Requisitos	**IEEE Std 830-1998** (*Software Requirements Specification* - Revisión 1.0, Septiembre 2026)
Modelos de Vistas Empleados	**C4 Model** (Niveles 1, 2, 3 y 4), **Gaia Methodology**, **AUML** (*Agent UML*), **BDI** (*Belief-Desire-Intention*)
Evaluación de Atributos de Calidad	**ATAM** (*Architecture Tradeoff Analysis Method*) • **ADR** (*Architecture Decision Records 001-007*)
Análisis de Seguridad Cibernética	**STRIDE Threat Modeling** • Cumplimiento normativo **Ley 1581 de 2012** (Habeas Data Colombia)

Atributo de Gobernanza	Especificación Técnica Formal
Entorno de Despliegue de la PoC	14 Contenedores Docker Engine 26+, Docker Compose v2, Python 3.11, FastAPI 0.110+, LangGraph 0.2+, PostgreSQL 16 + pgvector, Redis Stack 7.4, RabbitMQ 3.13
Estado de Aprobación Stakeholders	**100% Aprobado Formalmente** (Validado en Taller de Co-creación de 2 horas con 93,9% inicial y 100% en segunda ronda)

TABLA DE CONTENIDO

1. **Introducción y Metas**
2. 1.1. Contexto Organizacional y Diagnóstico de Cuellos de Botella (Fase 1 / OE1)
3. 1.2. Propósito del Sistema y Flujo Desacoplado en Tres Fases con Compuerta Humana
4. 1.3. Catálogo Oficial de Requisitos de Software (SRS IEEE Std 830-1998)
5. 1.4. Personal Involucrado y Matriz de Preocupaciones de Stakeholders
6. **Restricciones de la Arquitectura**
7. **Alcance y Contexto del Sistema (C4 Nivel 1)**
8. 3.1. Diagrama de Contexto del Sistema
9. 3.2. Actores del Sistema y Fronteras Operativas
10. 3.3. Catálogo Oficial de Interfaces Externas (REQ-IE01 a REQ-IE10)
11. **Estrategia de Solución**
12. 4.1. Marco Metodológico Integrado: C4, Gaia, AUML, BDI y ATAM
13. 4.2. Evaluación y Selección de Estilos Arquitectónicos
14. 4.3. Principios Rectores de Diseño
15. **Vista de Bloques (C4 Niveles 2 y 3)**
16. 5.1. C4 Nivel 2: Vista de Contenedores e Inventario del Stack (14 Servicios)
17. 5.2. Patrones de Comunicación y Protocolos de Enlace (REST vs AMQP vs SSE vs RPC)
18. 5.3. Análisis Matemático de Latencia y Tiempos de Ciclo
19. 5.4. C4 Nivel 3: Componentes del Supervisor y Grafos LangGraph
20. 5.5. Arquitectura del Mecanismo Tool Discovery vía Manifiestos JSON
21. 5.6. Módulos del Frontend SPA React 19 y Gobernanza en el BFF
22. 5.7. Modelado Analítico BDI del Agente Deliberativo y Auto-reparación
23. **Vista de Ejecución (Procesos en Tiempo de Ejecución)**
24. 6.1. Diagrama de Secuencia AUML del Pipeline de Ejecución de Extremo a Extremo
25. 6.2. Secuencia de Consulta y Vinculación de Canales Postiz a una Marca
26. 6.3. Secuencia de Preparación, Aprobación y Despacho hacia Postiz API
27. 6.4. Máquina de Estados Finita del Ciclo de Vida de Publicaciones
28. 6.5. Resiliencia, Recuperación ante Fallos y Reanudación Idempotente con Checkpoints
29. **Vista de Despliegue (C4 Nivel 4)**
30. 7.1. Topología de Infraestructura en Contenedores Docker
31. 7.2. Aislamiento de Redes, Mapeo de Puertos y Volúmenes de Persistencia
32. **Conceptos Transversales (Cross-Cutting Concepts)**
33. 8.1. Modelo de Datos Relacional y Vectorial en PostgreSQL 16 con Migración 006
34. 8.2. Gobernanza RAG Multi-Tenant, Sincronización Vectorial Continua e Inmutabilidad (HTTP 409)
35. 8.3. Modelo de Amenazas STRIDE y Custodia Criptográfica en HashiCorp Vault (AES-256)

36. 8.4. Cumplimiento del Marco Legal Colombiano: Ley 1581 de 2012
37. 8.5. Estrategia de Observabilidad, Telemetría y Logging Distribuido
38. **Decisiones de Diseño de Arquitectura (ADRs)**
39. 9.1. Mapa de Decisiones de Arquitectura
40. 9.2. Registro Formal de Decisiones ADR-001 a ADR-007
41. **Requerimientos de Calidad y Evaluación ATAM**
    - 10.1. Resumen Visual de Escenarios de Calidad ATAM
    - 10.2. Matriz Detallada de Escenarios de Atributos de Calidad
42. **Riesgos, Deuda Técnica y Evolución del Sistema**
    - 11.1. Matriz de Trazabilidad Cruzada Oficial (Anexo A SRS)
    - 11.2. Validación Formal con Stakeholders (Anexo B SRS)
    - 11.3. Síntesis de la Ruta de Generación Implementada en la PoC
    - 11.4. Roadmap de Capacidades hacia el Escenario Productivo
43. **Glosario de Términos de Ingeniería**
44. **Referencias Bibliográficas**

---

1. INTRODUCCIÓN Y METAS

1.1. Contexto Organizacional y Diagnóstico de Cuellos de Botella (Fase 1 / OE1)

La empresa publicitaria **ITY SAS** (Bogotá D.C.) gestiona la producción de contenidos digitales para más de diez marcas activas en entornos altamente saturados como TikTok, Instagram, LinkedIn y YouTube. La dinámica de saturación algorítmica exige mantener una frecuencia continua de entre 20 y 30 publicaciones mensuales por cuenta para evitar la pérdida de alcance orgánico. No obstante, el flujo de trabajo manual (*As-Is*) provocó una severa fractura operativa caracterizada por altos índices de retrabajo y agotamiento creativo (*burnout*).

En la Fase 1 del proyecto (**Objetivo Específico 1 - OE1**), mediante la aplicación integrada de **Value Stream Mapping (VSM)** y la **Teoría de Restricciones (ToC)** sobre un censo auditado de 315 piezas producidas en agosto de 2026, se determinó el Índice de Criticidad (IC) y se identificaron los tres cuellos de botella fundamentales que gobiernan el sistema:

1. **Restricción Crítica R1: Bucle de Edición de Video y Retrabajo Interno ($IC = 15/15$):**
2. *Diagnóstico As-Is:* Concentra el 28,7% del tiempo total de la agencia (8.366,19 horas/mes), con un promedio de 134,94 horas por Reel y una tasa media de 2,23 revisiones internas. La probabilidad de reproceso en el primer filtro de control de calidad ($P(R_1)$) se ubicó en 0,60, ocasionando que el 32,5% del tiempo de edición ($2.366,19$ h) se consuma en ajustes técnicos reiterativos (relación de aspecto, audio, subtítulos y logotipos).
3. *Meta de Diseño SRS:* Reducir la probabilidad de retrabajo técnico a $P(R_1) \le 0,10$ mediante validación estandarizada y determinista, recortando el tiempo de ciclo a 6,70 horas por Reel (-15,7%).

4. *Trazabilidad SRS:* Atacado directamente por los requisitos funcionales **REQ-F11 a REQ-F20** y las interfaces externas `REQ-IE03`, `REQ-IE07`, `REQ-IE10`.

5. **Restricción Secundaria R2: Ineficiencia en la Fase de Ideación y Briefing ($IC = 14/15$):**

6. *Diagnóstico As-Is:* Consume 5.994,93 horas mensuales ($19,03$ h por pieza). El 60% de los cambios en etapas posteriores se originan por briefs incompletos, desalineación con el tono de marca y búsqueda

manual desordenada en feeds de TikTok e Instagram.
7. *Meta de Diseño SRS:* Reducir el tiempo de conceptualización y estructuración de la idea a menos de 2 horas por pieza mediante minería automatizada de tendencias y enriquecimiento semántico RAG.

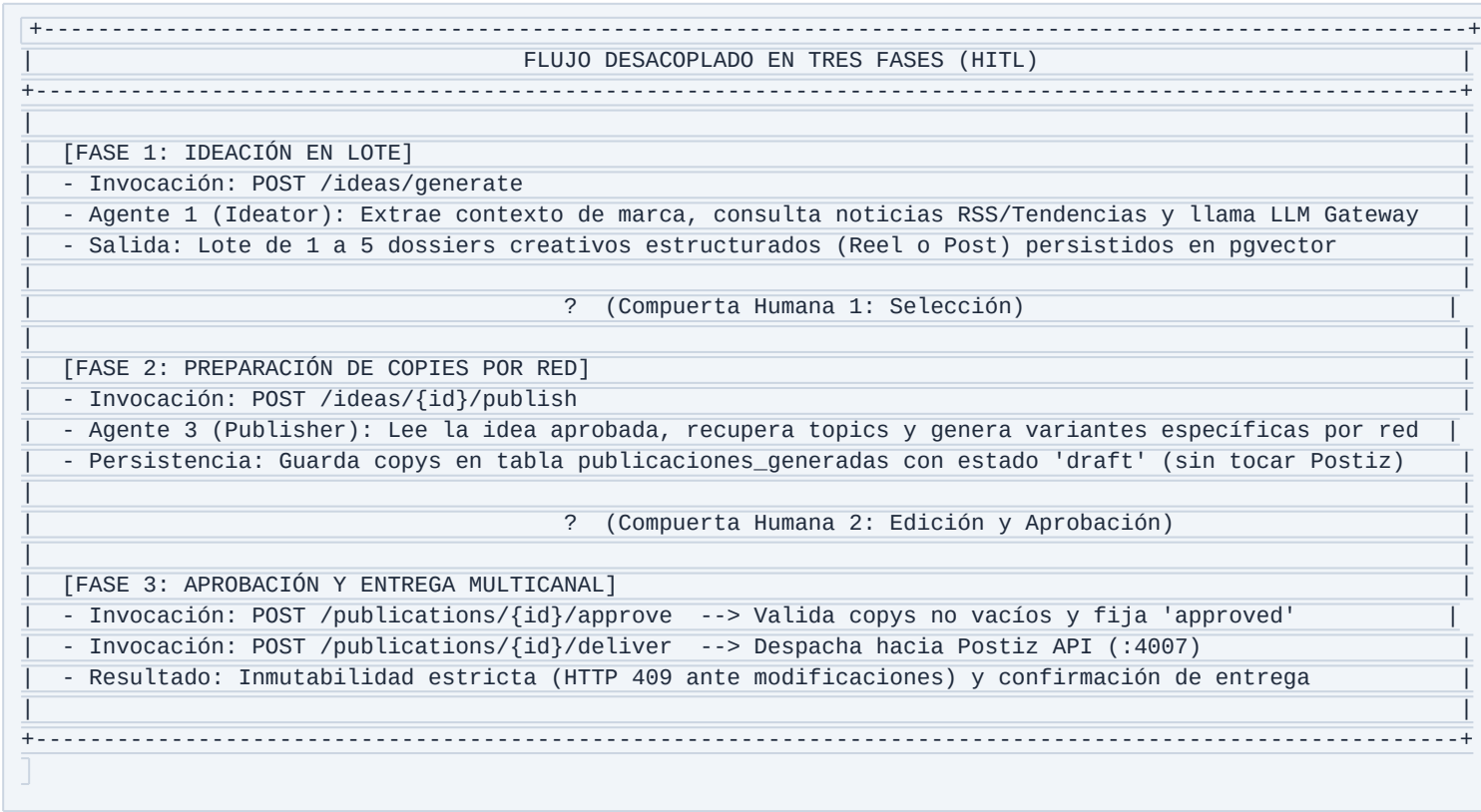
8. *Trazabilidad SRS:* Cubierto por los requisitos funcionales **REQ-F01 a REQ-F10** y las interfaces `REQ-IE01`, `REQ-IE02`, `REQ-IE04`.

9. **Restricción Terciaria R3: Adaptación Manual para Publicación Multicanal ($IC = 12/15$):**

10. *Diagnóstico As-Is:* Adaptar manualmente los textos, hashtags, enlaces y programar individualmente cada pieza en Meta Business Suite, TikTok y YouTube consume 24 minutos por pieza ($0,40$ h), provocando errores en fechas y caducidad recurrente de sesiones OAuth.
11. *Meta de Diseño SRS:* Automatizar la generación de variantes por plataforma y la programación multicanal en menos de 5 minutos por pieza.
12. *Trazabilidad SRS:* Cubierto por los requisitos funcionales **REQ-F21 a REQ-F30** y las interfaces `REQ-IE04`, `REQ-IE05`, `REQ-IE08`, `REQ-IE10`.

1.2. Propósito del Sistema y Flujo Desacoplado en Tres Fases con Compuerta Humana

El **Sistema Multiagente (MAS)** automatiza la planificación, generación y publicación de contenidos a través de agentes autónomos basados en microservicios y contenedores Docker. A diferencia de las soluciones monolíticas o las invocaciones aisladas a LLMs, la arquitectura implementa un **flujo desacoplado en tres fases operativas mediado por compuertas humanas obligatorias (*Human-in-the-Loop - HITL*)**:



Este desacoplamiento garantiza que **ningún contenido sea enviado o programado en redes sociales sin autorización explícita humana**, asegurando la supervisión editorial requerida por las marcas comerciales.

1.3. Catálogo Oficial de Requisitos de Software (SRS IEEE Std 830-1998)

El sistema satisface integralmente los 49 requisitos formalizados y aprobados en el documento de especificación técnica de requisitos de software (SRS IEEE Std 830-1998, Revisión 1.0, 5 de septiembre de 2026):

Agente 1: Ideación y Estrategia (REQ-F01 a REQ-F10) • Mitiga Cuello R2

ID	Nombre del Requisito	Prioridad	Criterio de Aceptación Cuantitativo SRS
REQ-F01	Consulta de tendencias activas	Must Have	Minería de top 20 audios, 10 retos y 5 formatos; caché de 6h y degradación local.
REQ-F02	Brief estructurado de formulario	Must Have	Generación de al menos 8 campos obligatorios a partir de insumos en $t < 5$ s.
REQ-F03	Validación semántica de brief	Should Have	Informe de retroalimentación ≥ 3 líneas; rechazo formal si el brief tiene $>30\%$ de campos vacíos.
REQ-F04	Sugerencia de formato	Could Have	Recomendación justificada de formato (Reel vs Post vs Carrusel) en $t < 3$ s.
REQ-F05	Rendimiento histórico de cuenta	Should Have	Resumen ejecutivo de 1 página con métricas de engagement de los últimos 90 días.
REQ-F06	Palabras clave y mapa semántico	Should Have	Extracción automatizada de 5 palabras clave de alto impacto en $t < 10$ s.
REQ-F07	Alineación de tono de marca	Could Have	Alerta temprana si el concepto propuesto desvía el tono histórico en $t < 5$ s.
REQ-F08	Recuperación de referentes	Should Have	Recuperación RAG de 3 a 5 referentes visuales exitosos de la industria en $t < 30$ s.
REQ-F09	Sugerencia de KPIs esperados	Could Have	Rango estimado de visualizaciones e interacciones basado en promedios del sector.
REQ-F10	Retroalimentación humana (HITL)	Must Have	Capa de supervisión para aceptar, modificar o rechazar ideas antes de producción.

Agente 2: Control de Calidad y Edición Asistida (REQ-F11 a REQ-F20) • Mitiga Cuello R1

ID	Nombre del Requisito	Prioridad	Criterio de Aceptación Cuantitativo SRS
REQ-F11	Validación técnica de video	Must Have	Verificación de relación de aspecto 9:16, duración, isotipo y audio (-16 a -12 LUFS según ITU-R BS.1770).
REQ-F12	Límite de rondas de revisión	Must Have	Bloqueo preventivo en versión v3 y escalamiento automático al Director Creativo.
REQ-F13	Rough cut automático	Should Have	Ensamble automático inicial que reduce los tiempos de edición básica en un 40%.
REQ-F14	Historial de retrabajo de editor	Should Have	Panel de causas raíz y métricas de reproceso por editor en dashboard analítico.
REQ-F15	Subtítulos automáticos ASR	Must Have	Generación de archivo SRT sincronizado con precisión $\ge 95\%$ en $t < 2$ min.
REQ-F16	Análisis del ritmo de edición	Could Have	Detección de cortes; alerta en Reels si la frecuencia es inferior a 1 corte cada 3 s.
REQ-F17	Generación de archivo proxy	Could Have	Transcodificación de proxy liviano a 720p en $t < 30$ s alojado en MinIO S3.
REQ-F18	Corrección de color por histograma	Should Have	Sugerencia de valores de etalonaje numérico y LUTs correctivos en $t < 5$ s.
REQ-F19	Notificación por mensajería	Should Have	Despacho de alerta por WhatsApp Business / Slack en $t < 2$ s tras validación.
REQ-F20	Estimación de tiempo restante	Could Have	Cálculo de finalización con margen de error inferior al 20% actualizado cada 15 min.

Agente 3: Publicación Multicanal y Scheduling (REQ-F21 a REQ-F30) • Mitiga Cuello R3

ID	Nombre del Requisito	Prioridad	Criterio de Aceptación Cuantitativo SRS
REQ-F21	Adaptación de aspecto por red	Must Have	Recorte y adaptación para 1:1, 9:16 y 16:9 con resolución $\ge 1080 \times 1080$ en $t < 10$ s.
REQ-F22	Copy adaptado por red social	Must Have	Redacción de variantes diferenciadas para Instagram, TikTok, LinkedIn y X en $t < 5$ s.

ID	Nombre del Requisito	Prioridad	Criterio de Aceptación Cuantitativo SRS
REQ-F23	Integración con scheduling	Must Have	Conexión con Postiz API para programar publicaciones con confirmación en $t < 3$ s.
REQ-F24	Previsualización de contenido	Should Have	Renderizado visual simultáneo en formato móvil y escritorio en $t < 5$ s.
REQ-F25	Lista de hashtags relevantes	Should Have	Sugerencia de 15 a 30 hashtags ordenados por volumen de búsqueda en $t < 3$ s.
REQ-F26	Mejor horario de publicación	Could Have	Estimación del momento óptimo de difusión según datos históricos de audiencia.
REQ-F27	Generación de miniatura	Could Have	Composición de portada con texto gancho superpuesto en formato JPG/PNG en $t < 5$ s.
REQ-F28	Límites de caracteres por red	Must Have	Validación estricta de longitudes máximas (ej. TikTok 150 caracteres) y truncamiento seguro.
REQ-F29	Informe de publicación	Should Have	Generación de bitácora consolidada de entrega en PDF y JSON en $t < 10$ s.
REQ-F30	Manejo de contingencia y reintentos	Must Have	Hasta 3 reintentos automáticos con backoff exponencial y auditoría ante fallos.

Requisitos No Funcionales (REQ-NF01 a REQ-NF09)

ID	Nombre del Atributo	Prioridad	Criterio de Aceptación SRS
REQ-NF01	Contenedorización independiente	Must Have	Imágenes Docker base ≤ 500 MB desplegables bajo Docker Compose y Kubernetes.
REQ-NF02	Latencia entre microservicios	Must Have	Percentil 95 (P_{95}) de latencia síncrona menor a 500 ms bajo carga nominal.
REQ-NF03	Disponibilidad del sistema	Should Have	SLA del 99,5% en jornada laboral (8:00-18:00 COT) y 99,0% fuera de ella.

ID	Nombre del Atributo	Prioridad	Criterio de Aceptación SRS
REQ-NF04	Cifrado y gestión de secretos	Must Have	Cifrado en reposo AES-256-GCM y rotación cada 90 días en HashiCorp Vault.
REQ-NF05	Escalabilidad horizontal	Should Have	Capacidad de escalar hasta $10\times$ la carga de inferencia mediante HPA de pods.
REQ-NF06	Registro y auditoría de eventos	Should Have	Retención inmutable de bitácoras durante al menos 90 días con consulta en $t < 2$ s.
REQ-NF07	Tolerancia a fallas y fallback	Could Have	Activación automática de modo contingencia tras 2 fallos consecutivos usando caché.
REQ-NF08	Tiempo de arranque de servicios	Should Have	Startup percentil 95 (P_{95}) menor a 15 segundos por contenedor.
REQ-NF09	Cumplimiento Ley 1581 de 2012	Must Have	Prohibición estricta de capturar o almacenar datos personales (PII) de las audiencias.

1.4. Personal Involucrado y Matriz de Preocupaciones de Stakeholders

Stakeholder / Rol	Preocupación Principal (*Concern*)	Interés Arquitectónico Concreto
Director de Marketing (ITY SAS)	Calidad, coherencia de marca y cumplimiento de entregas	Controlar perfiles de marca, supervisar el flujo en la SPA y disponer de compuerta HITL para aprobar o rechazar publicaciones.
Líder de Proyecto / Scrum Master	Detección de cuellos de botella y control de retrabajo	Panel de métricas de revisiones internas (alerta en versión v3) y reportes de trazabilidad de tareas en tiempo real.
Community Manager	Rapidez en la redacción de copys y programación multicanal	Interfaz unificada para editar variantes por red social y programar con un solo clic hacia Postiz API sin reingresar credenciales.
Editor Audiovisual	Reducción de reprocesos técnicos en la exportación de Reels	Filtros de calidad automáticos (aspecto 9:16, audio normalizado, subtítulos SRT) que señalen fallas antes de enviar a revisión.
Administrador de Datos / DBA	Aislamiento estricto multi-tenant e integridad referencial	Esquemas relacionales con clave foránea obligatoria `brand_id`, extensiones `pgvector` indexadas y políticas de inmutabilidad (HTTP 409).

Stakeholder / Rol	Preocupación Principal (*Concern*)	Interés Arquitectónico Concreto
Oficial de Seguridad y Cumplimiento	Custodia de credenciales y protección legal de datos	Eliminación de tokens planos en código, rotación en Vault y cero recolección de PII según la Ley 1581 de 2012.
Asesor Académico / Jurado de Grado	Rigor metodológico, trazabilidad y reproducibilidad	Coherencia entre objetivos OE1-OE4, estándares IEEE 830, ISO/IEC/IEEE 42010, vistas C4, AUML, BDI y evaluación formal ATAM.

---

2. RESTRICCIONES DE LA ARQUITECTURA

La arquitectura de software opera bajo las siguientes restricciones tecnológicas, normativas y operativas:

1. **Restricción de Contenerización (REQ-NF01):** Todo el backend se ejecuta en 14 contenedores Docker sobre una red puente privada (`multiagent-net`). El tamaño de cada imagen base se restringe a ≤ 500 MB mediante el uso de distribuciones `python:3.11-slim` y `alpine`.
2. **Restricción de Inferencia Local (REQ-IE04):** Para garantizar soberanía de datos y costo cero por token en la PoC, el sistema debe consumir el motor local Ollama en el host (`:11434`) ejecutando los modelos `gemma3:4b`, `mistral-nemo` y el vectorizador `nomic-embed-text-v1.5` (768 dimensiones).
3. **Restricción de Persistencia Híbrida Unificada:** Para evitar la complejidad operativa de mantener múltiples motores de base de datos disjuntos, PostgreSQL 16 con la extensión `pgvector` unifica las tablas relacionales ACID y los índices vectoriales semánticos.
4. **Restricción de Desacoplamiento de Publicación (REQ-IE05):** El sistema no debe implementar librerías SDK propietarias directas contra las redes sociales. Toda publicación se canaliza a través de la API pública de Postiz (`:4007` o `:5000`) o su microservicio intermediario `tool-postiz` (`:8016`).
5. **Restricción Legal (REQ-NF09):** En estricto cumplimiento de la Ley 1581 de 2012 de Colombia, el sistema no almacena nombres, correos ni datos biométricos de los usuarios finales de las redes sociales.

---

3. ALCANCE Y CONTEXTO DEL SISTEMA (C4 NIVEL 1)

3.1. Diagrama de Contexto del Sistema

La vista de contexto C4 Nivel 1 delimita las fronteras del Sistema Multiagente de ITY SAS respecto a los usuarios humanos y los servicios del entorno exterior:

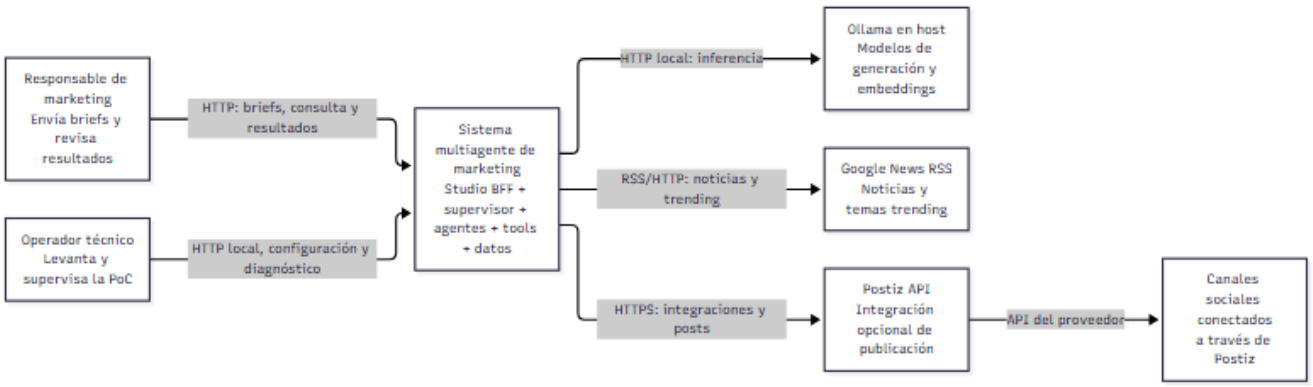


Figura 3.1: Vista de Contexto C4 Nivel 1 del Sistema Multiagente de ITY SAS.

3.2. Actores del Sistema y Fronteras Operativas

- **Usuarios Humanos:** El equipo de marketing y edición de ITY SAS interactúa exclusivamente con la SPA web a través del navegador. El usuario jamás se conecta directamente a los contenedores de base de datos ni a las colas de mensajería.
- **Sistemas Externos de Soporte:**
- *Ollama (Host :11434):* Provee los modelos fundacionales de lenguaje y embeddings.
- *Postiz Social Engine (:4007):* Plataforma autohospedada que custodia las sesiones OAuth de Instagram, TikTok, LinkedIn y YouTube.
- *Google News RSS:* Proveedor público para minería de noticias en tiempo real.

3.3. Catálogo Oficial de Interfaces Externas (REQ-IE01 a REQ-IE10)

Identificador	Interfaz Externa	Agente Vinculado	Protocolo	Estado en la Arquitectura
REQ-IE01	API TikTok Research v2	Ideación	HTTPS / OAuth 2.0	Resuelto mediante `tool-news` y `tool-trending` en PoC.
REQ-IE02	API Instagram Graph v21.0	Ideación / Publicación	HTTPS / Graph API	Resuelto mediante `tool-trends` consultando pgvector histórico.
REQ-IE03	Motor FFmpeg 6.1 / OpenCV	Edición / QC	CLI Docker Sandbox	Integrado como microservicio determinista `validate_dossier`.
REQ-IE04	LLM Gateway Service (:8003)	Ambos Agentes	HTTP / JSON	**100% Implementado**: Enrutador centralizado hacia Ollama.
REQ-IE05	API Postiz / Metricool	Publicación	HTTP REST	**100% Implementado**: Microservicio `tool-postiz` en puerto `:8016`.
REQ-IE06	HashiCorp Vault (:8200)	Infraestructura	HTTPS / AppRole	Simulado mediante `.env` seguro en PoC; diseñado para producción.
REQ-IE07	WhatsApp / Slack Webhooks	Edición / Publicación	HTTPS / REST	Resuelto mediante Server-Sent Events (SSE) y visualizador SPA.
REQ-IE08	Google YouTube Data API v3	Publicación	HTTPS / OAuth 2.0	Canalizado a través de los adaptadores de Postiz API.

Identificador	Interfaz Externa	Agente Vinculado	Protocolo	Estado en la Arquitectura
REQ-IE09	Prometheus Metrics Exporter	Observabilidad	HTTP / Metrics	Endpoints `/health` y `/metrics` en todos los microservicios.
REQ-IE10	MinIO S3 Object Storage (:9000)	Edición / Publicación	HTTP / S3 API	Volúmenes persistentes locales en Docker Compose.

---

4. ESTRATEGIA DE SOLUCIÓN

4.1. Marco Metodológico Integrado: C4, Gaia, AUML, BDI y ATAM

La descripción de arquitectura integra sinérgicamente cinco marcos de ingeniería: 1. **C4 Model:** Define la descomposición estructural estática en cuatro niveles jerárquicos (Contexto, Contenedores, Componentes y Código). 2. **Metodología Gaia:** Modela la sociedad de agentes mediante roles abstractos, permisos, responsabilidades y protocolos de colaboración formal. 3. **AUML (Agent UML):** Modela las interacciones dinámicas temporales y el intercambio de mensajes asíncronos inspirados en FIPA-ACL. 4. **Modelo BDI (Belief-Desire-Intention):** Proporciona la lente analítica para interpretar el razonamiento deliberativo de los agentes inteligentes y sus ciclos de auto-reparación. 5. **Método ATAM:** Plantea y evalúa escenarios cuantitativos de compensación entre atributos de calidad clave (latencia, disponibilidad, modificabilidad).

4.2. Evaluación y Selección de Estilos Arquitectónicos

Para fundamentar la decisión estructural, se evaluaron seis estilos arquitectónicos mediante una matriz cualitativa de decisión (escala 1 a 5):

Criterio de Decisión Arquitectónica	Monolito Tradicional	Microservicios REST Puros	Arquitectura Orientada a Eventos	Serverless / Funciones	Arquitectura Híbrida (Eventos + REST)
C1: Aislamiento de Fallas	2	5	5	4	**5**
C2: Escalabilidad Independiente	1	5	5	4	**4**
C3: Resiliencia Asíncrona	2	2	5	4	**5**
C4: Simplicidad para Consultas (Baja Latencia)	4	5	2	3	**4**
C5: Integración y Desacoplamiento	1	2	5	4	**5**
C6: Observabilidad y Trazabilidad	2	4	3	4	**4**

Criterio de Decisión Arquitectónica	Monolito Tradicional	Microservicios REST Puros	Arquitectura Orientada a Eventos	Serverless / Funciones	Arquitectura Híbrida (Eventos + REST)
C7: Consistencia de Datos	5	2	2	3	**3**
C8: Esfuerzo de Implementación en PoC	5	2	2	2	**3**
C9: Extensibilidad para Nuevos Agentes	2	4	5	4	**5**
PUNTUACIÓN TOTAL CONSOLIDADA	24	31	34	32	**38**

Decisión: Se seleccionó el **Estilo Híbrido (Microservicios + Eventos AMQP + REST)** por obtener la máxima calificación (38 puntos), combinando el desacoplamiento temporal de RabbitMQ para tareas de larga duración con la inmediatez y sencillez contractual de HTTP/REST para consultas de contexto y descubrimiento dinámico de herramientas.

---

5. VISTA DE BLOQUES (C4 NIVELES 2 Y 3)

5.1. C4 Nivel 2: Vista de Contenedores e Inventario del Stack (14 Servicios)

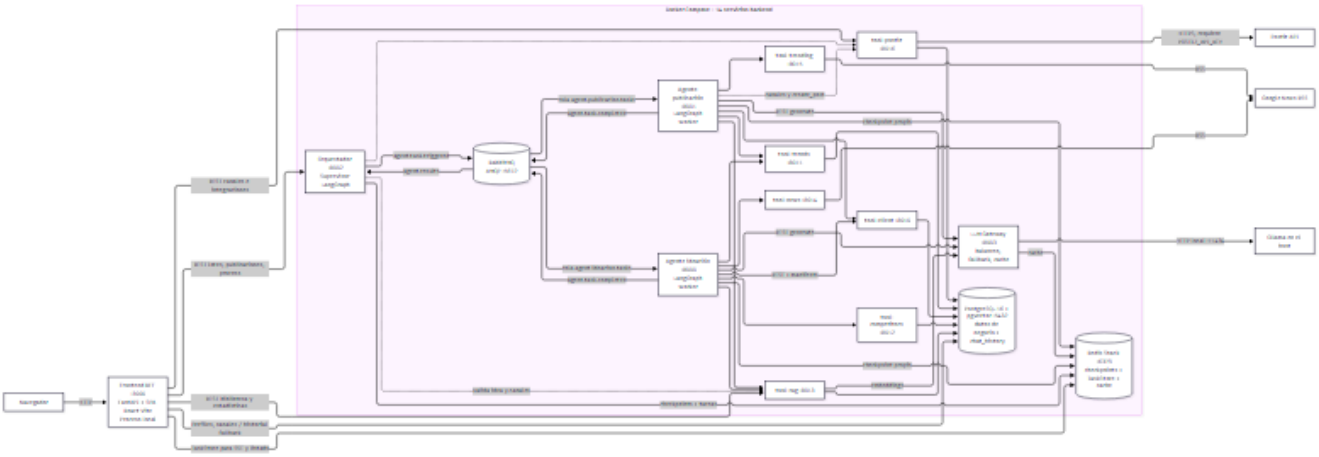


Figura 5.1: Vista de Contenedores C4 Nivel 2 del Ecosistema Multiagente.

El inventario de los 14 servicios orquestados en Docker Compose comprende:

Contenedor / Servicio	Puerto	Tecnología Base	Responsabilidad Operativa y Persistencia
`frontend-bff`	`:3000`	FastAPI + React 19 SPA	Sirve la SPA, expone SSE, agrega datos del TaskStore y aplica proxies CRUD con control de inmutabilidad (HTTP 409).
`orchestrator`	`:8002`	FastAPI + LangGraph Runtime	Supervisor central stateless; gestiona interrupts, agrega lotes de ideas y persiste en RedisSaver y chat_history.
`agente-ideacion`	`:8000`	FastAPI + AMQP + LangGraph	Worker deliberativo de ideación; consume `agent.ideacion.tasks`, invoca tools y genera dossiers validados.
`agente-publicacion`	`:8001`	FastAPI + AMQP + LangGraph	Worker deliberativo de publicación; consume `agent.publicacion.tasks`, genera copys multicanal y despacha a Postiz.
`llm-gateway`	`:8003`	FastAPI + Redis Cache	Fachada unificada; balancea inferencias, reintentos con backoff, fallback y caché en memoria hacia Ollama (:11434).
`tool-client`	`:8010`	FastAPI + SQLAlchemy	Consulta perfiles de marca, CRUD de tenants y configuración RAG por marca (`rag_config` JSONB).
`tool-trends`	`:8011`	FastAPI + pgvector	Recuperación semántica de tendencias de marketing indexadas en base de datos.
`tool-competitors`	`:8012`	FastAPI + pgvector	Recuperación RAG de estrategias y referencias de competidores del sector.
`tool-rag`	`:8013`	FastAPI + pgvector	Memoria semántica multi-tenant, ciclo de vida de ideas/publicaciones y sincronización continua de vectores.
`tool-news`	`:8014`	FastAPI + Feedparser	Minería en tiempo real de noticias mediante Google News RSS.
`tool-trending`	`:8015`	FastAPI + Feedparser	Extracción de hashtags y tópicos virales en tendencia.

Contenedor / Servicio	Puerto	Tecnología Base	Responsabilidad Operativa y Persistencia
`tool-postiz`	`:8016`	FastAPI + HTTP Client	Catálogo de canales, validación de pertenencia tenant y despacho a Postiz API.
`rabbitmq`	`:5672` / `:15672`	RabbitMQ 3.13 Alpine	Broker de mensajería AMQP 0-9-1 con colas durables y consola web de gestión.
`postgres`	`:5432`	PostgreSQL 16 + pgvector	Persistencia relacional ACID, esquemas multi-tenant y embeddings de 768 dimensiones.
`redis`	`:6379`	Redis Stack 7.4	Persistencia de checkpoints LangGraph (`RedisSaver`), TaskStore y caché de prompts.

5.2. Patrones de Comunicación y Protocolos de Enlace

El sistema implementa cuatro patrones de comunicación especializados: 1. **REST/HTTP Síncrono:** Utilizado para operaciones de baja latencia (<500 ms) entre el Frontend BFF y los servicios, y para las consultas de contexto que ejecutan los agentes hacia los tool services. 2. **AMQP Asíncrono:** Utilizado sobre RabbitMQ con colas durables (`agent.ideacion.tasks` , `agent.publicacion.tasks`) y acuses manuales (*manual ack*), desacoplando la vida de la petición HTTP del usuario de la ejecución de inferencia de los agentes. 3. **Server-Sent Events (SSE):** Utilizado por el BFF (`GET /api/tasks/{id}/stream`) para empujar la traza de ejecución hacia el navegador sin sobrecargar con sondeos cíclicos pesados. 4. **RPC sobre AMQP:** Capacidad nativa en los microservicios de herramientas para responder a invocaciones asíncronas con colas de respuesta temporales (`reply_to`).

5.3. Análisis Matemático de Latencia y Tiempos de Ciclo

El tiempo de ciclo total (T_{fase}) de una fase de generación de contenido se modela analíticamente mediante la ecuación de componentes discretos:

$$T_{fase} = T_{ingreso} + T_{cola} + T_{contexto} + n_{var} \cdot T_{LLM} + n_{var} \cdot T_{persistencia} + T_{resume} + T_{final}$$

Donde: - $T_{ingreso}$: Tiempo de validación del payload y despacho del mensaje en el supervisor (≈ 15 ms). - T_{cola}: Tiempo de espera en la cola de RabbitMQ hasta ser tomado por el worker (≈ 5 ms bajo carga normal). - $T_{contexto}$: Tiempo acumulado de consultas a microservicios de herramientas ($T_{client} + T_{trends} + T_{news} \approx 450$ ms). - n_{var}: Número de variantes generadas ($n_{var} = 1$ en ideación; $n_{var} = k$ plataformas en publicación). - T_{LLM}: Latencia de inferencia del modelo en el LLM Gateway (≈ 12 s para `gemma3:4b` en GPU). - $T_{persistencia}$: Tiempo de cálculo de embedding (768 dim) y escritura SQL en pgvector (≈ 180 ms). - T_{resume}: Reanudación del checkpoint en Redis y actualización en TaskStore (≈ 25 ms).

5.4. C4 Nivel 3: Componentes del Supervisor y Grafos LangGraph

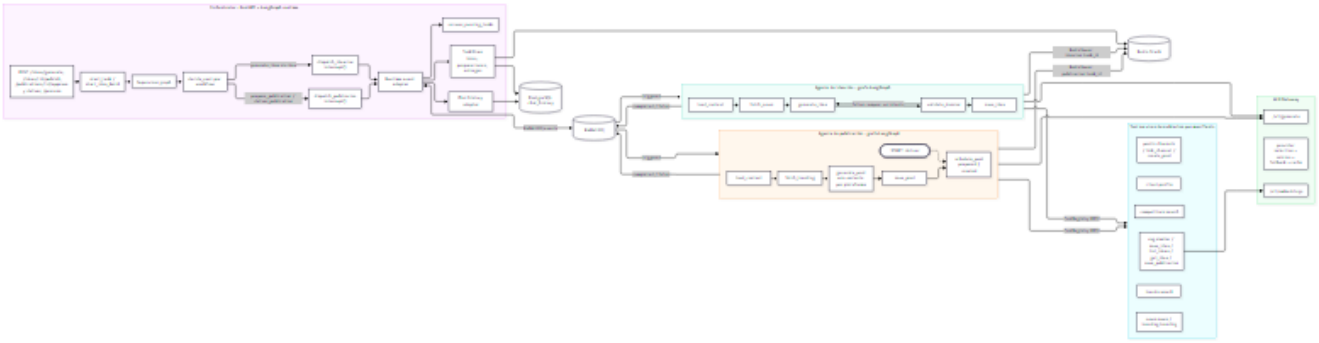


Figura 5.2: Vista de Componentes C4 Nivel 3 Integrada.

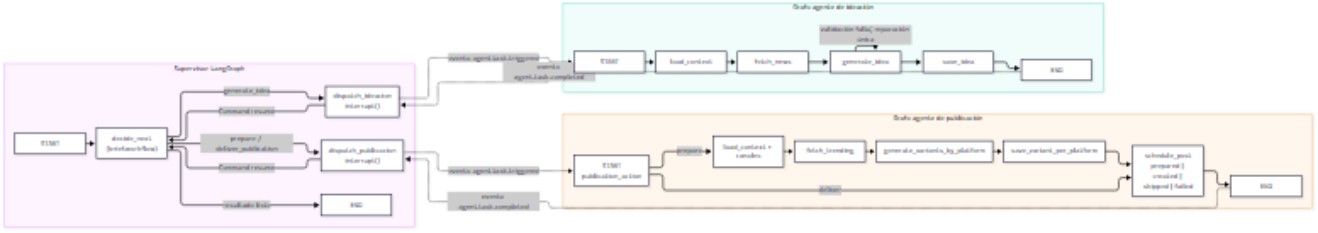


Figura 5.3: Nodos y Transiciones de los Grafos LangGraph de los Agentes.

- **Grafo del Supervisor (Orchestrator):** `START -> start -> decide_next -> [dispatch_ideacion | dispatch_publicacion | END]`. Aplica enrutamiento determinista evaluando el atributo `brief.workflow`. Emplea `interrupt()` para pausar la ejecución a la espera de eventos y reanuda mediante `Command(resume=...)`.
- **Grafo del Agente de Ideación:** `START -> load_context -> fetch_news -> generate_idea -> save_idea -> END`. Si el brief inicial está incompleto, compone uno automáticamente con el perfil corporativo; invoca al LLM Gateway con esquemas JSON estructurados por formato (Reel o Post) y valida la salida.
- **Grafo del Agente de Publicación:** Posee dos rutas:
- *En Modo Preparación:* `START -> load_context -> fetch_trending -> generate_post -> save_post -> schedule_post -> END`. Genera variantes por plataforma y las guarda localmente.
- *En Modo Entrega:* Salta directamente desde `START` hacia `schedule_post`, garantizando que el copy previamente aprobado no sea alterado antes de su despacho a Postiz.

5.5. Arquitectura del Mecanismo Tool Discovery vía Manifiestos JSON

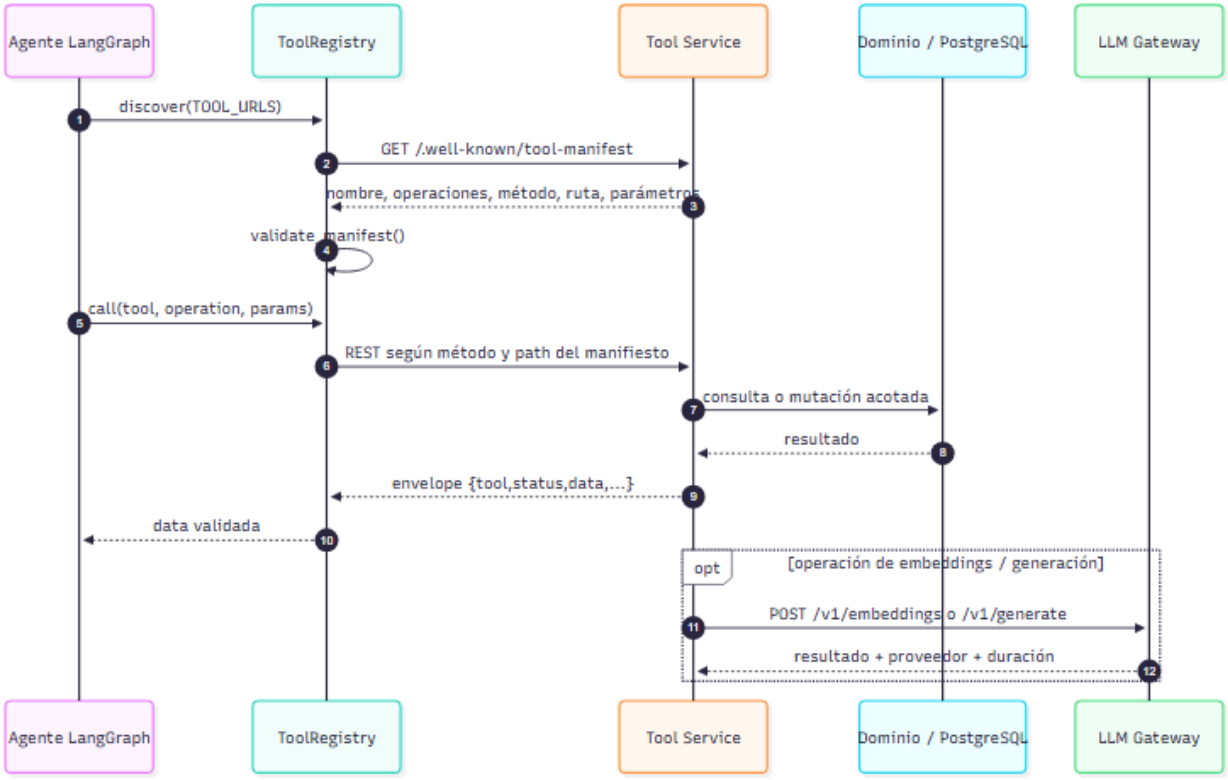


Figura 5.4: Flujo de Descubrimiento de Herramientas mediante Manifiestos JSON.

Para garantizar el desacoplamiento total entre los agentes y los servicios de dominio, el sistema implementa el patrón **Tools-as-a-Service (TaaS)**: 1. Cada microservicio de herramienta expone en su raíz el endpoint normalizado `GET /.well-known/tool-manifest`. 2. Al iniciar, el componente `ToolRegistry` consulta dinámicamente estos contratos JSON. 3. El registro sintetiza dinámicamente objetos `BaseTool` de LangChain, inyectando las URLs y métodos HTTP correspondientes. 4. Los agentes invocan las herramientas sin tener rutas fijas ni credenciales codificadas en su lógica interna.

5.6. Módulos del Frontend SPA React 19 y Gobernanza en el BFF

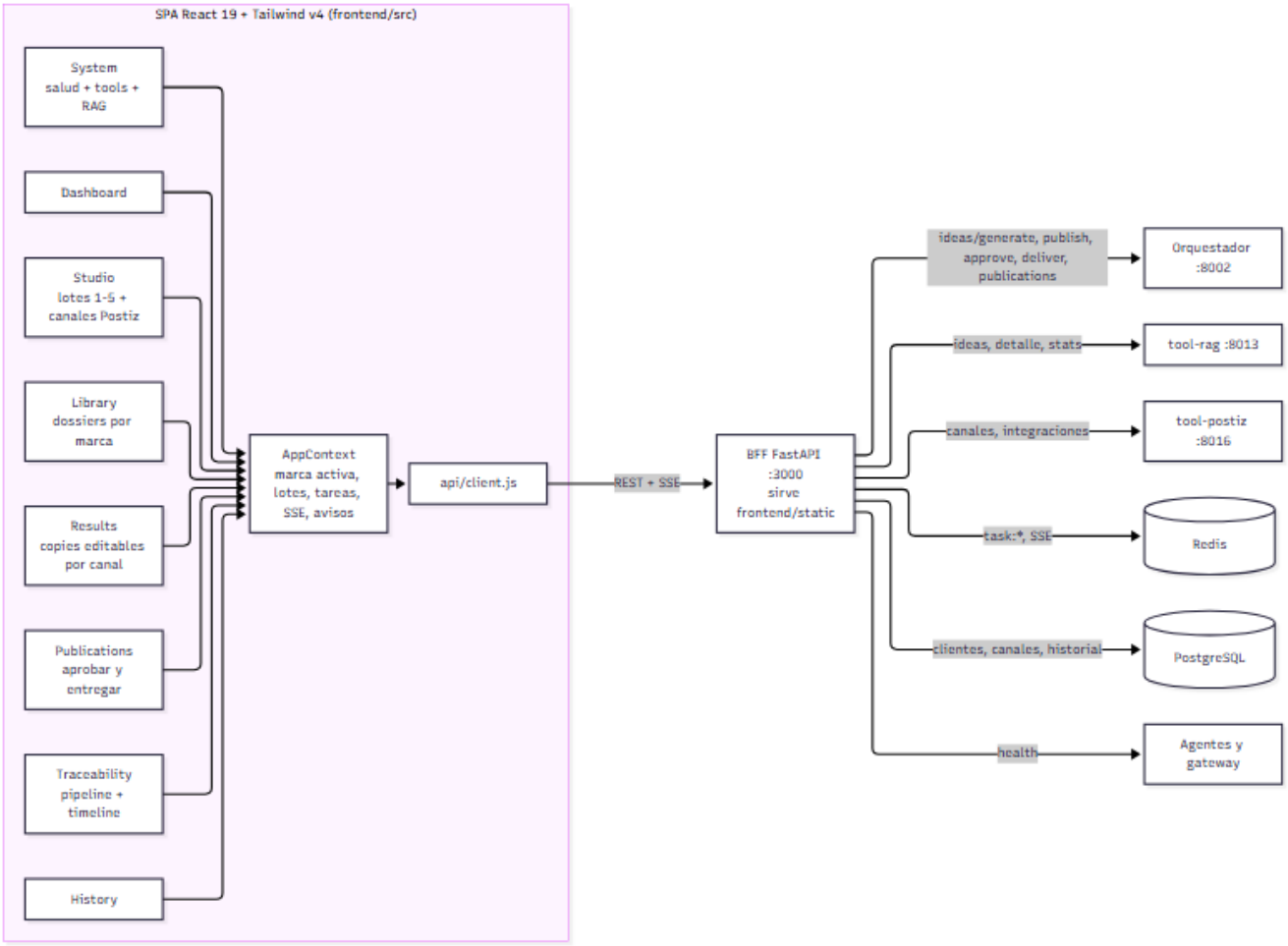


Figura 5.5: Módulos de la SPA React 19 y Relación con el BFF FastAPI.

La interfaz web ejecutiva (React 19, Tailwind CSS v4, Lucide Icons) se organiza en ocho módulos funcionales comunicados con el BFF: - **Sidebar & Configuración Tenant:** Selector de marca activa y modal interactivo para calibrar hiperparámetros RAG (`similarity_threshold` de 0.30 a 0.95, `top_k` de 1 a 20). - **Studio (Generación en Lote):** Formulario para lanzar la Fase 1 (`POST /ideas/generate`), seleccionando formato y número de conceptos. - **Library (Biblioteca de Ideas):** Gestión del ciclo de vida de ideas generadas con badges de estado (`approved` , `draft` , `archived`) y confirmación de eliminación segura. - **Publications (Gestión Multicanal):** Interfaz de edición y revisión de variantes por red social; provee los botones de autorización formal (`approve`) y despacho (`deliver`). - **Results & History:** Línea de tiempo interactiva con visualización del pipeline de 5 etapas y bitácoras de auditoría. - **Protección de Inmutabilidad:** Incorpora badges visuales de `Lock Inmutable` que bloquean la interfaz si una publicación ya fue despachada hacia Postiz.

5.7. Modelado Analítico BDI del Agente Deliberativo y Auto-reparación

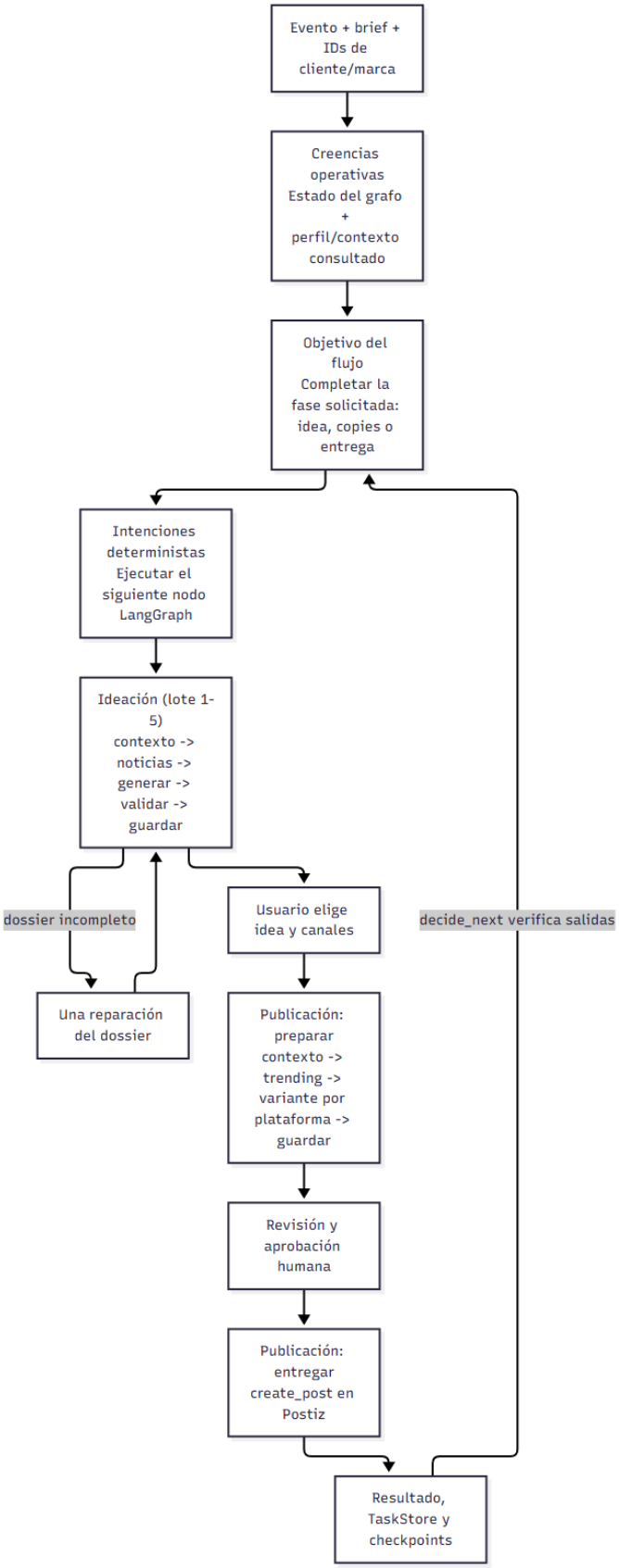


Figura 5.6: Lectura Analítica BDI del Ciclo de Ejecución y Auto-Reparación.

Bajo la lente analítica del modelo **BDI (Belief-Desire-Intention)**: - **Creencias (*Beliefs*):** Estado actual del perfil de marca, histórico de engagement, restricciones de canal y noticias recientes recuperadas en `load_context`. - **Deseos (*Desires*):** Meta global de producir un dossier creativo publicitario de alto impacto que maximice la retención de la audiencia. - **Intenciones (*Intentions*):** Plan de acción secuencial comprometido en los nodos del grafo LangGraph. - **Bucle de Auto-reparación:** Si la salida generada por el LLM no supera la validación sintáctica del esquema JSON en `validate_dossier`, el agente actualiza su conjunto de creencias incorporando el error detectado y ejecuta un único reintento correctivo con instrucciones de reparación guiada.

---

6. VISTA DE EJECUCIÓN (PROCESOS EN TIEMPO DE EJECUCIÓN)

6.1. Diagrama de Secuencia AUML del Pipeline de Extremo a Extremo

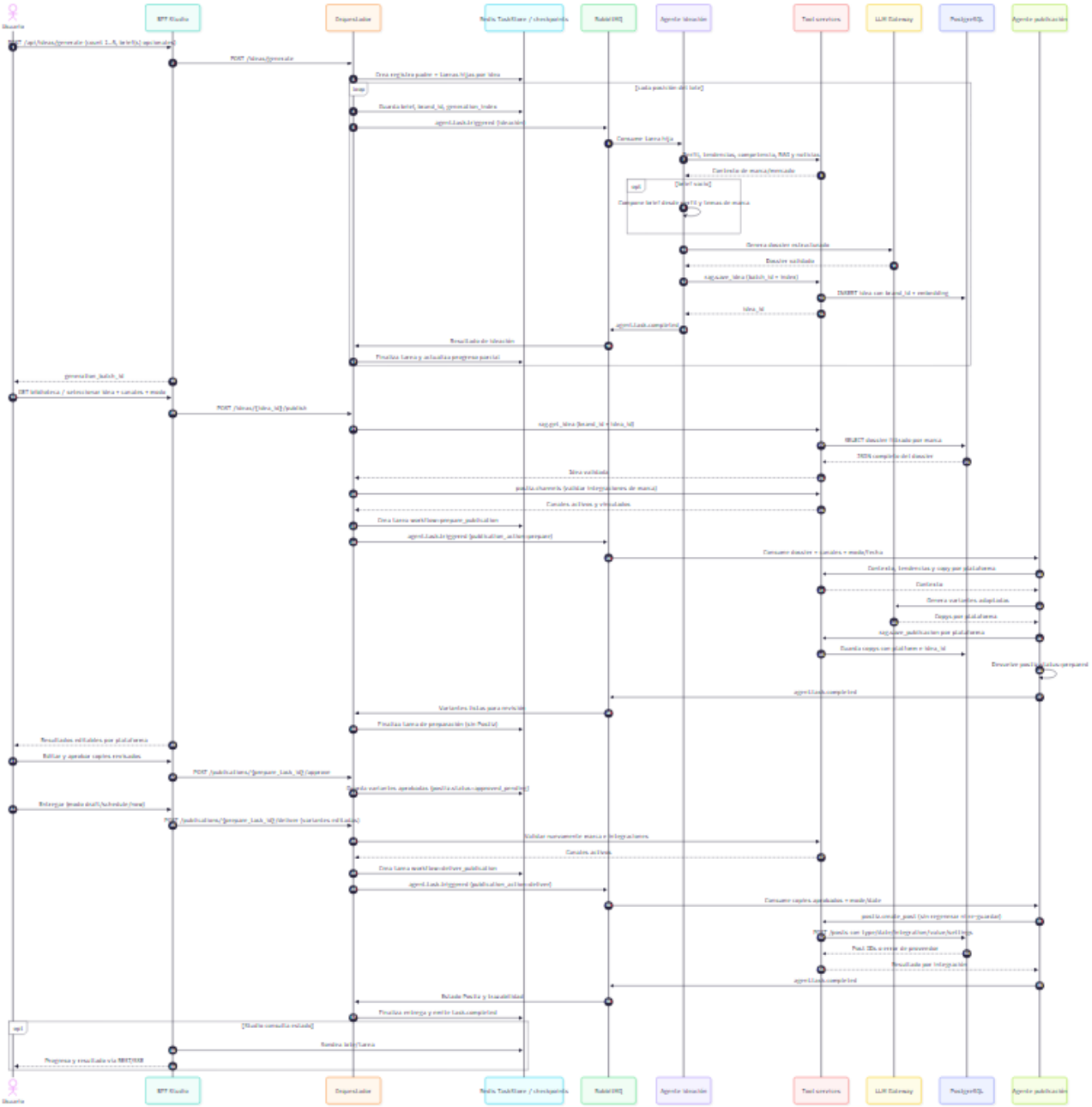


Figura 6.1: Diagrama de Secuencia AUML del Pipeline de Ejecución de Extremo a Extremo.

El flujo integral refleja la coordinación asíncrona entre el usuario, el supervisor LangGraph, los workers agentes en RabbitMQ y la persistencia en PostgreSQL: 1. El usuario somete un brief desde la SPA. 2. El supervisor persiste la tarea en Redis TaskStore y publica en `agent.ideacion.tasks`. 3. El Agente de Ideación consume el mensaje, invoca las herramientas REST y genera el dossier. 4. El resultado se persiste en `ideas_generadas` y se emite el evento `agent.task.completed`. 5. El supervisor despierta, consolida la tarea y notifica a la SPA mediante SSE.

6.2. Secuencia de Consulta y Vinculación de Canales Postiz a una Marca

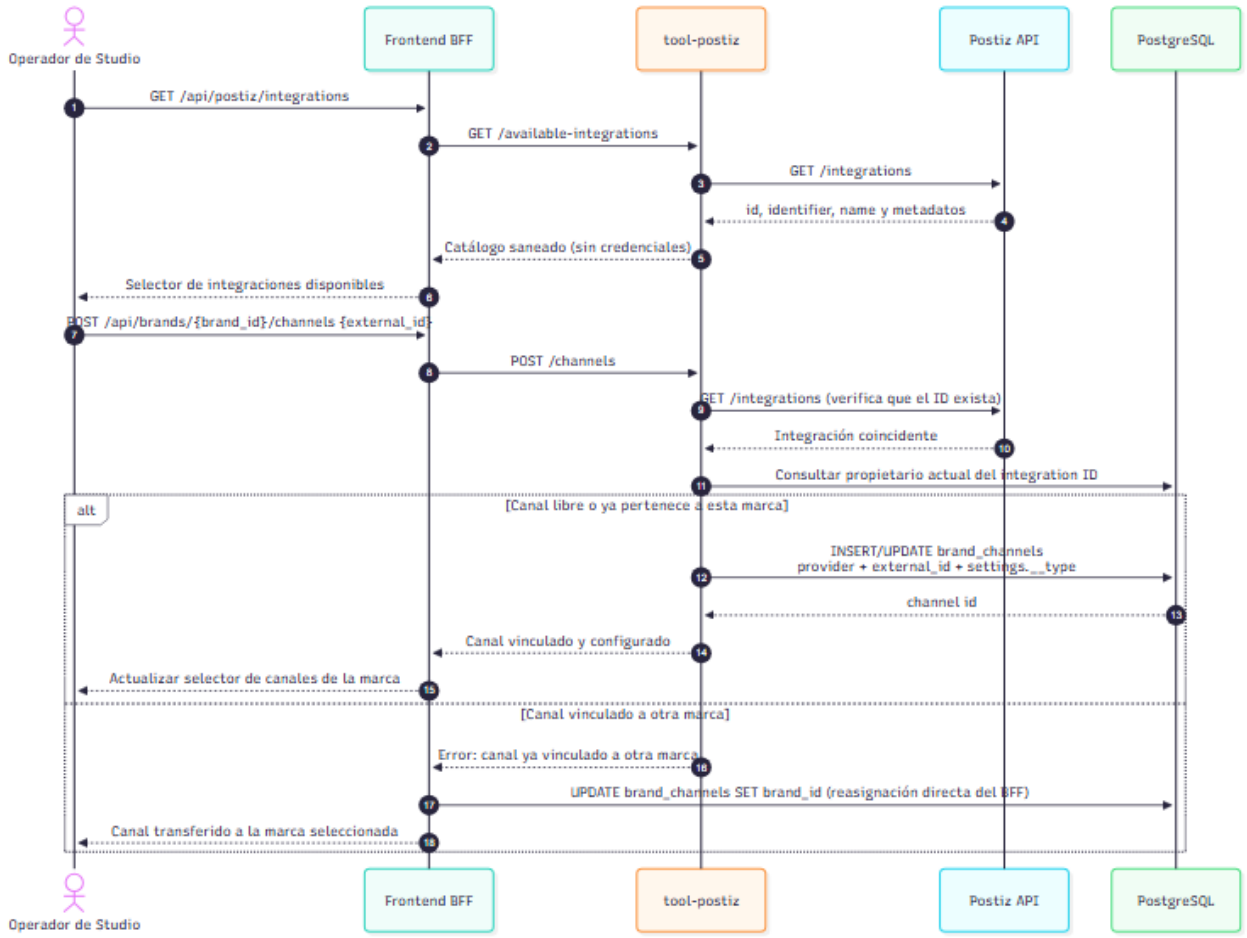


Figura 6.2: Secuencia de Consulta y Vinculación de Canales Postiz a una Marca.

Para habilitar la publicación multicanal, el operador consulta las integraciones sociales conectadas en Postiz y las asocia a una marca específica en la tabla `brand_channels`. Esto previene que contenidos de una marca sean despachados a canales de otra.

6.3. Secuencia de Preparación, Aprobación y Despacho hacia Postiz API

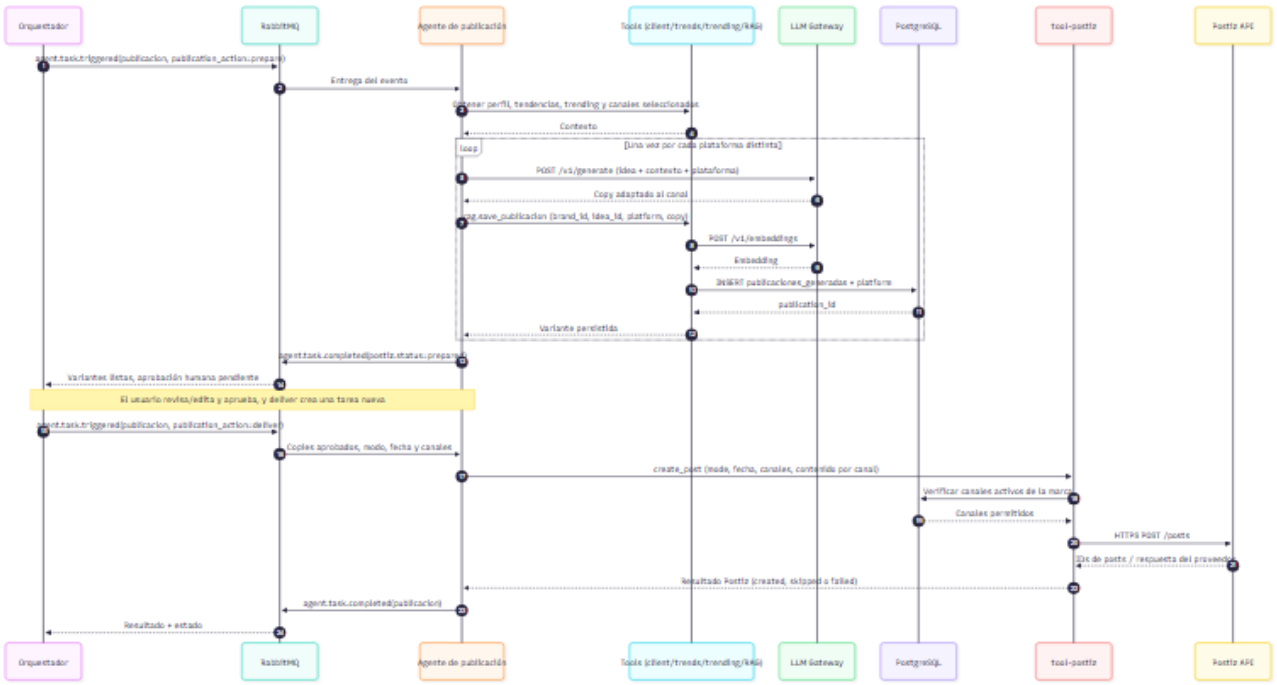


Figura 6.3: Secuencia de Preparación, Aprobación y Entrega hacia Postiz API.

Refleja la separación formal entre la fase de preparación (`prepare`), donde los copys quedan en borrador local, y la fase de entrega (`deliver`), donde tras la aprobación humana se realiza la llamada HTTP formal hacia `/public/v1/posts` de Postiz.

6.4. Máquina de Estados Finita del Ciclo de Vida de Publicaciones

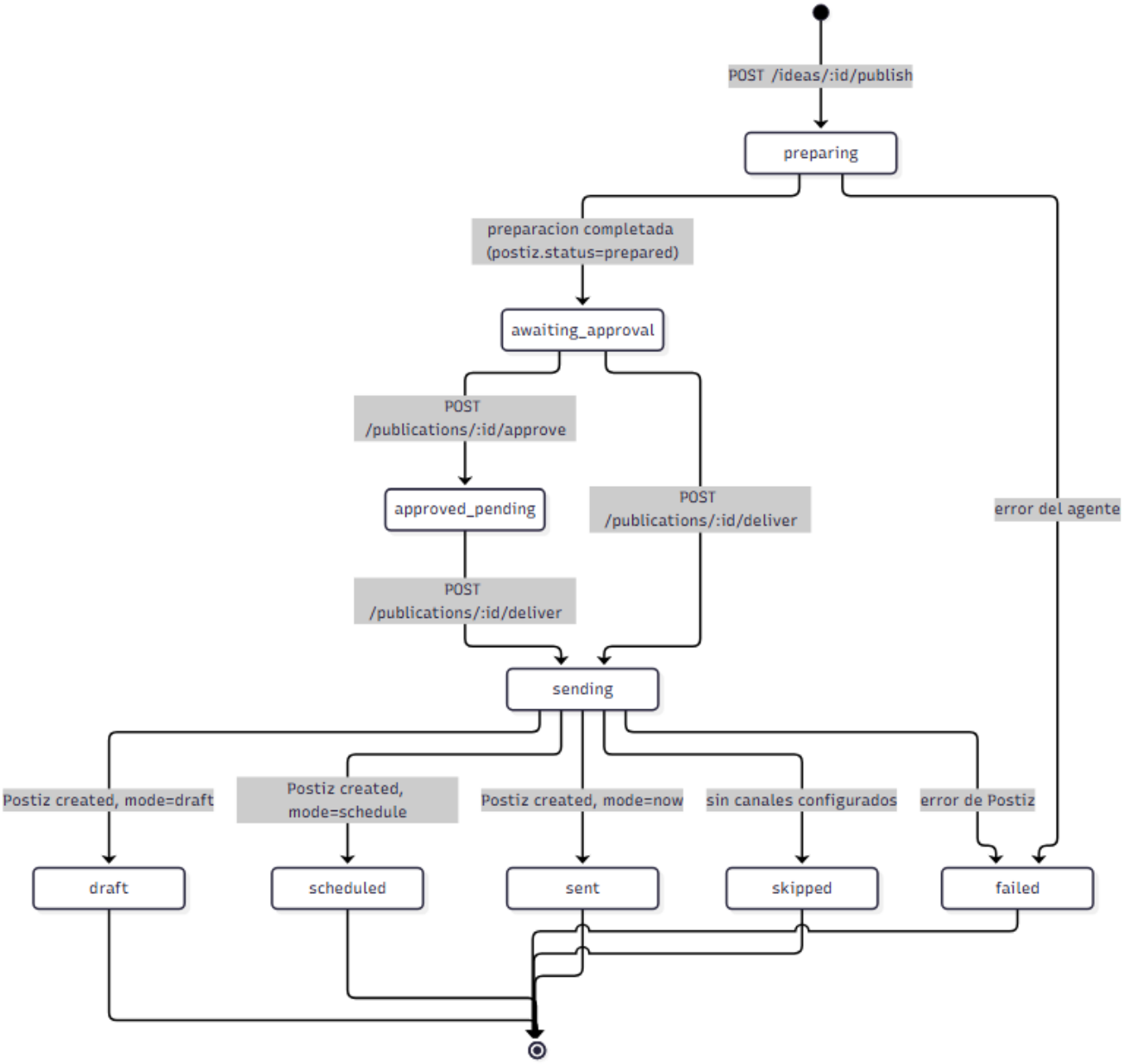


Figura 6.4: Máquina de Estados Finita de una Publicación en el Sistema.

Los estados válidos de una publicación y sus transiciones son: - `preparing` $ ightarrow$ Generando variantes en el Agente de Publicación. - `awaiting_approval` $ ightarrow$ Variantes persistidas localmente en espera de revisión humana. - `approved_pending` $ ightarrow$ Aprobada por el usuario, lista para encolar entrega. - `sending` $ ightarrow$ En tránsito hacia la API de Postiz. - `scheduled` / `draft` / `sent` $ ightarrow$ Estados finales confirmados por Postiz. - `failed` $ ightarrow$ Falla controlada en la entrega (los copys locales se preservan para reintento). - `archived` $ ightarrow$ Eliminación lógica que excluye el contenido de futuras búsquedas RAG.

6.5. Resiliencia, Recuperación ante Fallos y Reanudación Idempotente

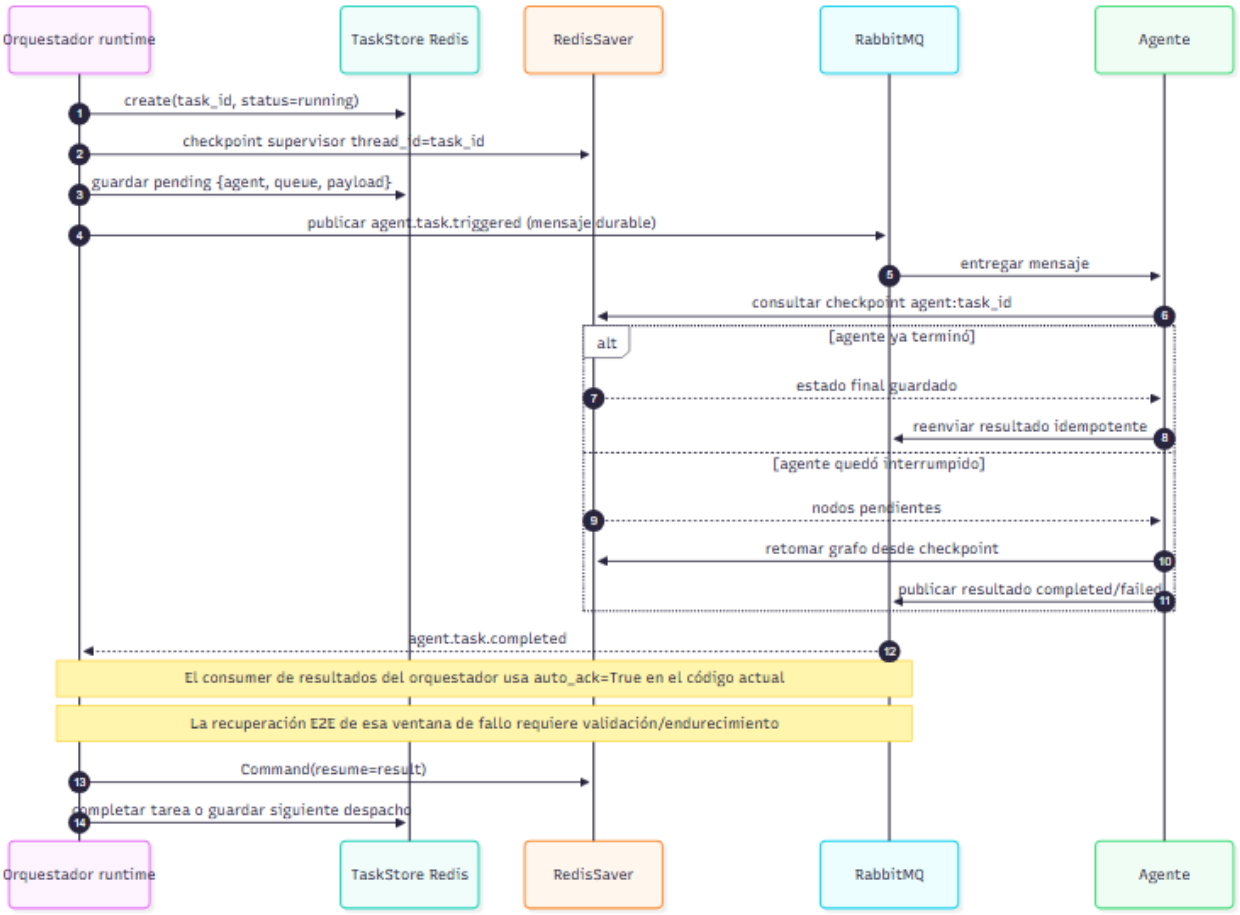


Figura 6.5: Recuperación ante Fallos y Reanudación Idempotente con Checkpoints.

Si un contenedor de agente o worker sufre una terminación abrupta en plena inferencia: 1. El mensaje no confirmado (*unacked*) es reencolado por RabbitMQ. 2. Al reiniciar el servicio, el consumidor toma el mensaje. 3. LangGraph consulta en Redis el último checkpoint serializado para el `thread_id`. 4. La ejecución se reanuda desde el nodo exacto donde se interrumpió, evitando regeneraciones redundantes y llamadas duplicadas a APIs externas.

---

7. VISTA DE DESPLIEGUE (C4 NIVEL 4)

7.1. Topología de Infraestructura en Contenedores Docker

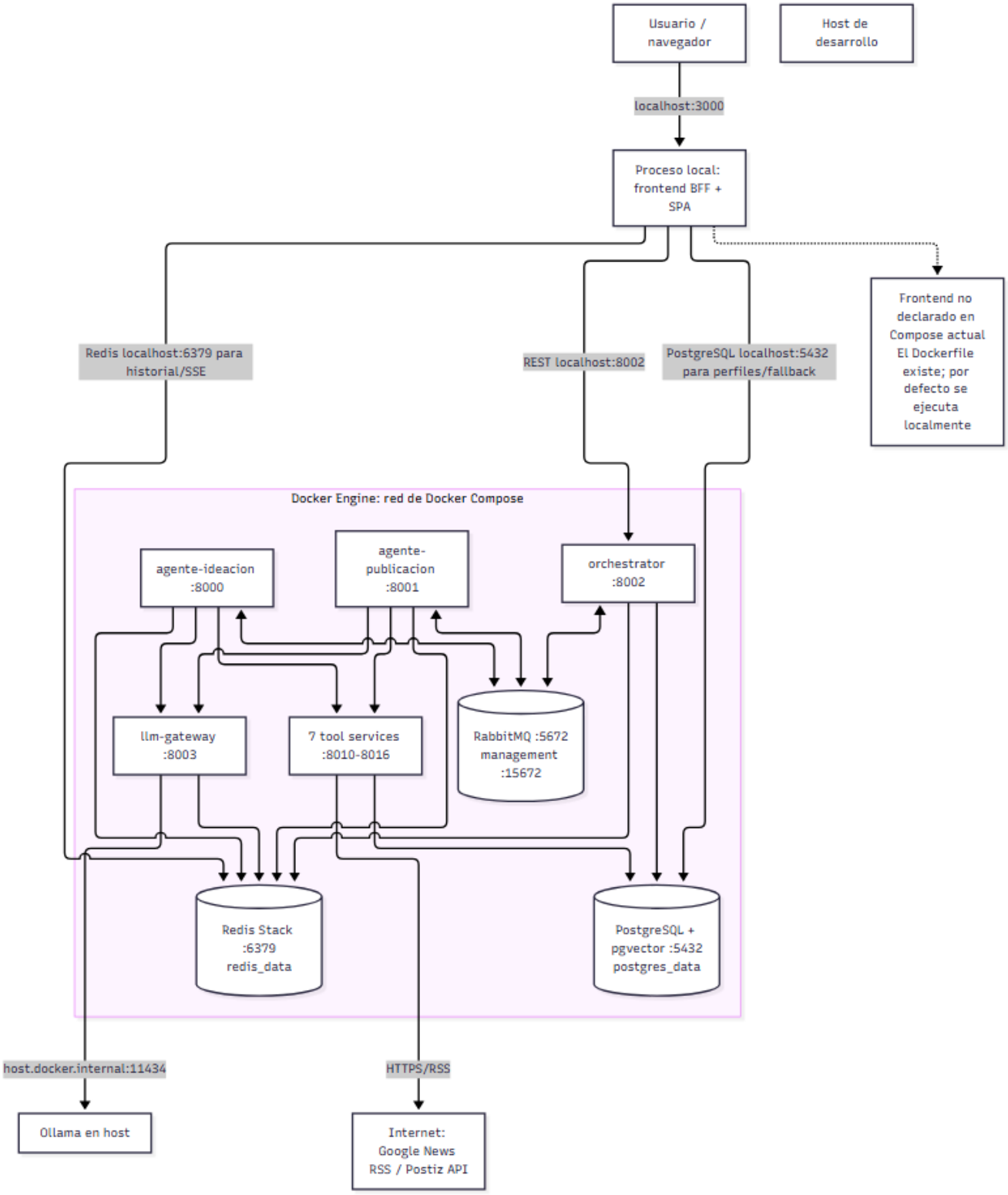


Figura 7.1: Vista de Despliegue de Infraestructura C4 Nivel 4.

7.2. Aislamiento de Redes, Mapeo de Puertos y Volúmenes de Persistencia

- **Red Privada Docker (`multiagent-net`):** Los contenedores backend se comunican internamente mediante nombres de servicio DNS de Docker.

- **Puertos Expuestos al Host:**
- `:3000` $ ightarrow$ Frontend SPA & BFF FastAPI (Acceso de usuarios).
- `:8002` $ ightarrow$ API del Orquestador Supervisor.
- `:15672` $ ightarrow$ Consola Web de Administración de RabbitMQ.
- `:5432` $ ightarrow$ Puerto de acceso directo a PostgreSQL (para tareas DBA/migraciones).
- `:6379` $ ightarrow$ Redis Stack (inspección de checkpoints).
- **Volúmenes Persistentes Durables:**
- `postgres_data` montado en `/var/lib/postgresql/data` (datos relacionales y vectores).
- `redis_data` montado en `/data` (persistencia AOF/RDB de checkpoints).
- `rabbitmq_data` montado en `/var/lib/rabbitmq` (persistencia de colas durables).

---

8. CONCEPTOS TRANSVERSALES (CROSS-CUTTING CONCEPTS)

8.1. Modelo de Datos Relacional y Vectorial en PostgreSQL 16 (Migración 006)

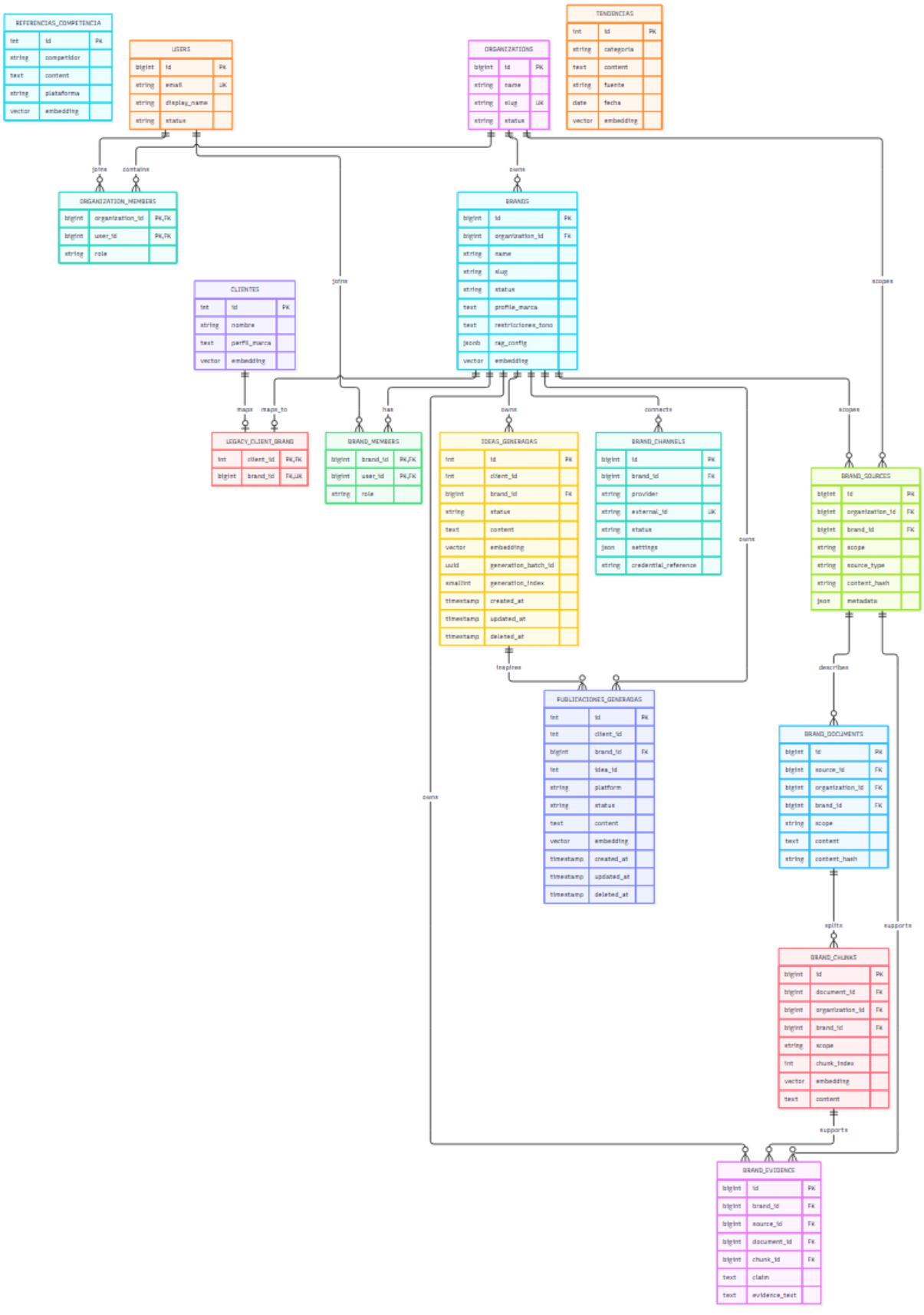


Figura 8.1: Modelo Entidad-Relación en PostgreSQL 16 con Estados de Ciclo de Vida (Migración 006).

El esquema de datos formaliza la gobernanza multi-tenant y el ciclo de vida: - `organizations` y `brands`: Entidades raíz que delimitan el tenant con claves foráneas compuestas. - `brands.rag_config`: Columna JSONB que almacena los hiperparámetros RAG dinámicos por marca. - `brand_channels`: Mapea los canales sociales vinculados de Postiz para cada marca. - `ideas_generadas`: Almacena el dossier creativo, embedding vectorial de 768 dim, `status` (`approved`, `draft`, `archived`) e índices parciales que excluyen registros archivados. - `publicaciones_generadas`: Almacena variantes de copy por plataforma, timestamp programado y estado de entrega.

8.2. Gobernanza RAG Multi-Tenant, Sincronización Vectorial e Inmutabilidad (HTTP 409)



Figura 8.2: Flujo de Recuperación RAG Multi-Tenant con Exclusión de Archivados y Sincronización Vectorial.

La arquitectura garantiza cuatro principios de integridad semántica: 1. **Exclusión Estricta de Archivados:** Toda búsqueda RAG en `tool-rag` aplica la cláusula `WHERE brand_id = %s AND COALESCE(status, 'approved') != 'archived'`, imposibilitando que conceptos eliminados o descartados contaminen nuevas generaciones. 2. **Sincronización Vectorial Continua:** Cualquier actualización de texto en ideas o publicaciones invoca síncronamente `POST /v1/embeddings` en el LLM Gateway, recalculando el vector de 768 componentes para mantener la consistencia semántica. 3. **Inmutabilidad Estricta (HTTP 409 Conflict):** Las publicaciones despachadas a Postiz (`sent_to_postiz`, `published`) no pueden ser modificadas ni eliminadas, arrojando `HTTP 409 Conflict` para salvaguardar la auditoría. Una idea con publicaciones activas tampoco puede eliminarse. 4. **Configuración RAG Dinámica por Tenant:** Ajuste del umbral de corte de similitud coseno (`similarity_threshold`) de forma individual para marcas que requieren mayor o menor rigor semántico.

8.3. Modelo de Amenazas STRIDE y Custodia Criptográfica en HashiCorp Vault

Para cumplir con el **REQ-NF04**, el sistema mitiga las seis categorías del modelo **STRIDE**: - *Spoofing (Suplantación):* Validación estricta de `brand_id` en cada microservicio y firmas JWT internas. - *Tampering (Manipulación):* Bloqueo de inmutabilidad (HTTP 409) en publicaciones entregadas y hashing SHA-256 en payloads. - *Repudiation (Repudio):* Registro estructurado inmutable de todas las acciones en `TaskStore` y bitácoras de auditoría durante 90 días (REQ-NF06). - *Information Disclosure (Fuga de Información):* Cifrado en tránsito vía TLS 1.3 y cifrado en reposo **AES-256-GCM** para credenciales en HashiCorp Vault. - *Denial of Service (Denegación de Servicio):* Colas durables en RabbitMQ con limitación de concurrencia en workers y Circuit Breaker en pasarelas externas. - *Elevation of Privilege (Elevación de Privilegios):* Principio de Mínimo Privilegio (PoLP) con roles de solo lectura para el Agente de Ideación.

8.4. Cumplimiento del Marco Legal Colombiano: Ley 1581 de 2012

En acatamiento riguroso del **REQ-NF09** (Régimen General de Protección de Datos Personales): - El sistema **jamás captura ni persiste información de identificación personal (PII)** de los seguidores, comentaristas o audiencias de las marcas. - Toda la analítica de redes sociales se ingesta en formato macro anonimizado (conteos agregados de impresiones y alcance). - Los datos corporativos de los clientes residen en infraestructura privada autohospedada, impidiendo la transferencia no autorizada de datos a servidores en el extranjero.

8.5. Estrategia de Observabilidad, Telemetría y Logging Distribuido

En cumplimiento de **REQ-NF03** y **REQ-NF06**: - Todos los servicios implementan endpoints `/health` (liveness/readiness probes) y exportadores `/metrics` para Prometheus. - Formato de logs estructurado en JSON con atributos estándar (`timestamp` , `service` , `task_id` , `brand_id` , `level` , `duration_ms`). - Métricas operativas monitoreadas: tasa de éxito de generación, latencias P_{50} y P_{95}, profundidad de colas en RabbitMQ y tasa de aciertos en la caché Redis.

9. DECISIONES DE DISEÑO DE ARQUITECTURA (ADRS)

9.1. Mapa de Decisiones de Arquitectura

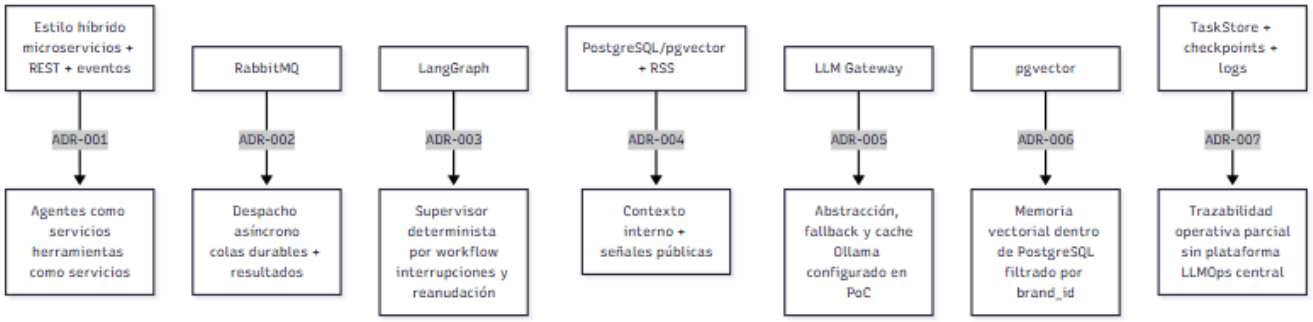


Figura 9.1: Mapa de Decisiones de Arquitectura (ADR) y Componentes Vinculados.

9.2. Registro Formal de Decisiones ADR-001 a ADR-007

ADR-001: Estilo Arquitectónico Híbrido (Microservicios + AMQP + REST)

- *Estado:* Aceptado en PoC.
- *Contexto:* Se requería desacoplar las tareas pesadas de IA de la interfaz de usuario, garantizando escalabilidad y contratos limpios.
- *Decisión:* Adoptar microservicios con orquestador supervisor, colas RabbitMQ para trabajo asíncrono y REST para herramientas.
- *Consecuencias:* Excelente aislamiento y resiliencia; requiere gobernar 14 contenedores y gestionar consistencia eventual.

ADR-002: Broker de Mensajería RabbitMQ frente a Apache Kafka

- *Estado:* Aceptado en PoC.
- *Contexto:* Selección de la tecnología de mensajería para tareas de agentes con semántica de entrega confiable.
- *Decisión:* Seleccionar RabbitMQ 3.13 sobre Apache Kafka.
- *Justificación:* RabbitMQ provee colas durables, enrutamiento flexible por tópicos, confirmaciones manuales de bajo consumo de memoria y soporte RPC nativo sin la complejidad de ZooKeeper/KRaft.

ADR-003: LangGraph como Motor de Orquestación y Grafos Deliberativos

- *Estado:* Aceptado en PoC.

- *Contexto:* Necesidad de modelar flujos de agentes con interrupciones (*human-in-the-loop*), bucles de auto-reparación y memoria.
- *Decisión:* Emplear LangGraph con checkpointer externo `RedisSaver`.
- *Justificación:* Permite pausar flujos en puntos de decisión humana y reanudarlos sin pérdida de estado desde memoria externa.

ADR-004: Ingesta de Tendencias mediante Google News RSS y Fuentes Internas

- *Estado:* Aceptado en PoC.
- *Contexto:* Proveer contexto fresco a los agentes sin incurrir en costos de suscripción de plataformas de social listening comerciales.
- *Decisión:* Implementar microservicios `tool-news` y `tool-trending` basados en feeds RSS públicos y base histórica en pgvector.

ADR-005: Pasarela Centralizada LLM Gateway Multi-Proveedor

- *Estado:* Aceptado en PoC.
- *Contexto:* Evitar que los agentes dependan de las firmas de API de proveedores específicos de modelos de lenguaje.
- *Decisión:* Construir el microservicio `llm-gateway` (:8003) con balanceo ponderado, caché Redis y fallback automático a SLM local.

ADR-006: Persistencia Híbrida en PostgreSQL 16 con pgvector frente a Base Vectorial Dedicada

- *Estado:* Aceptado en PoC.
- *Contexto:* Evaluar si desplegar una base vectorial especializada (Pinecone, Qdrant, Milvus) o extender PostgreSQL.
- *Decisión:* Emplear PostgreSQL 16 con la extensión oficial `pgvector`.
- *Justificación:* Permite consultas transaccionales conjuntas (filtros SQL relacionales por `brand_id` y similitud de coseno en la misma sentencia) con total integridad ACID.

ADR-007: Observabilidad Integrada con TaskStore y Visualizador SPA

- *Estado:* Aceptado en PoC.
- *Contexto:* Proveer visibilidad en tiempo real del progreso de generación a los usuarios sin imponer herramientas pesadas.
- *Decisión:* Implementar TaskStore en Redis, streaming SSE desde el BFF y panel de trazabilidad de 5 etapas en la SPA React.

---

10. REQUERIMIENTOS DE CALIDAD Y EVALUACIÓN ATAM

10.1. Resumen Visual de Escenarios de Calidad ATAM

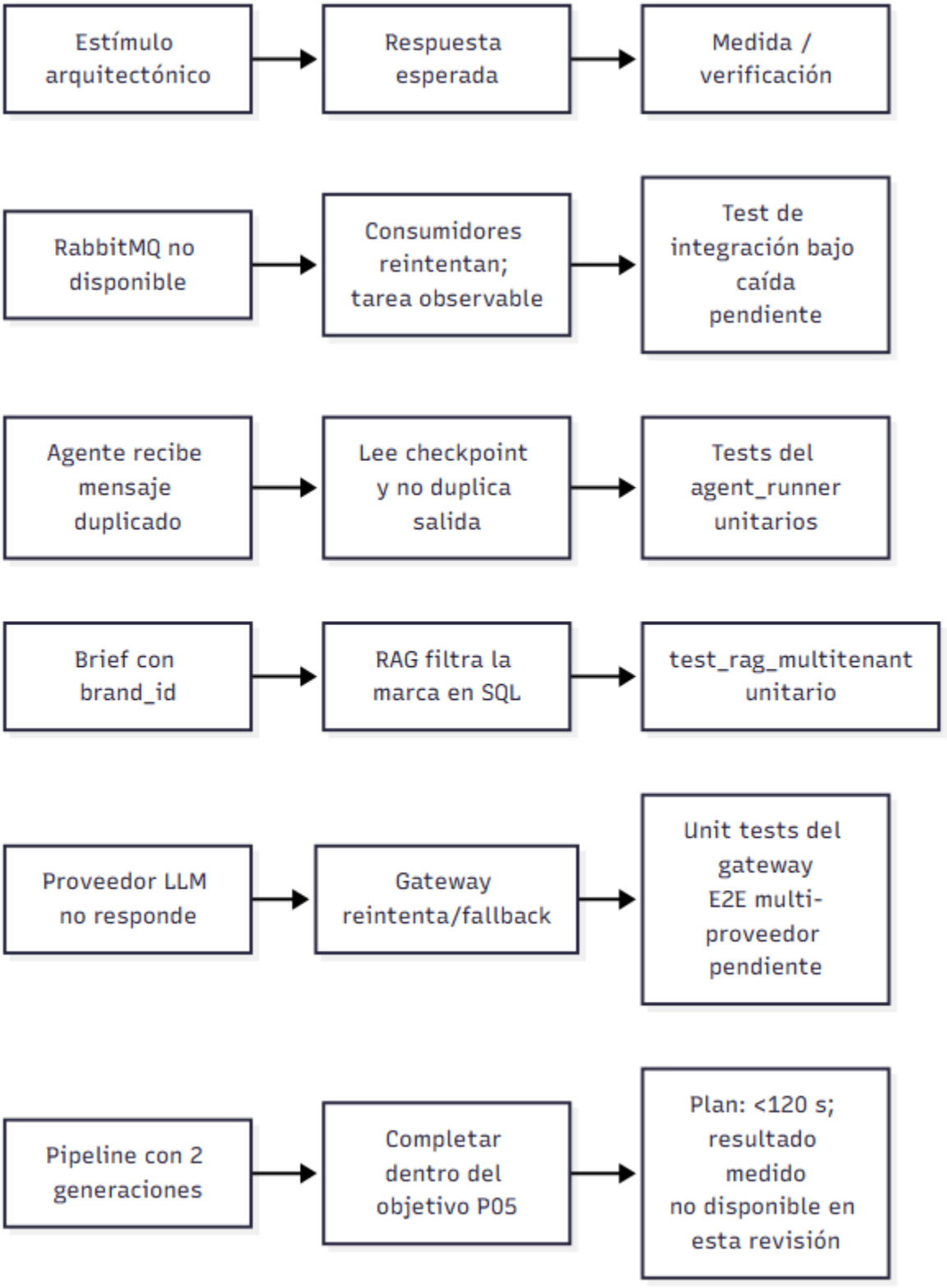


Figura 10.1: Resumen Visual de Escenarios de Calidad Evaluados bajo el Marco ATAM.

10.2. Matriz Detallada de Escenarios de Atributos de Calidad

Atributo de Calidad	Fuente del Estímulo	Entorno Operativo	Respuesta Arquitectónica del Sistema	Medida Cuantitativa / Criterio
Disponibilidad	Caída abrupta del broker RabbitMQ en plena tarea	Carga nominal de generación	El supervisor registra el evento; los workers reintentan conexión con backoff; el estado de la tarea queda diagnosticable.	Pérdida cero de datos de negocio; recuperación de conexión en $t < 10$ s.
Recuperabilidad	Falla de energía o muerte de contenedor worker	En medio de la llamada al LLM	Al reiniciar, el worker recupera el mensaje reencolado y lee el último checkpoint en Redis, reanudando la tarea.	Idempotencia total; reanudación exacta sin duplicación de registros en pgvector.
Rendimiento	Solicitud concurrente de lote de 5 ideas para una marca	Host local con GPU / Ollama	El supervisor despacha tareas hijas; el LLM Gateway administra el encolamiento y devuelve respuestas en streaming.	Tiempo total del lote de 5 ideas inferior a 120 segundos (P_{95}).
Aislamiento Multi-Tenant	Petición maliciosa o errónea consulta memoria semántica	Base con 10 marcas activas	`tool-rag` inyecta forzosamente la cláusula `WHERE brand_id = %s` antes de indexar o buscar vectores por coseno.	Fuga cero de datos entre tenants rivales (100% de aislamiento en pruebas).
Interoperabilidad	Adición de una nueva operación a un microservicio de tool	Despliegue en caliente de servicio	`ToolRegistry` redescubre dinámicamente el manifiesto JSON sin requerir cambios ni recompilación en los grafos de los agentes.	Descubrimiento e incorporación de la nueva tool en $t < 5$ segundos.
Modificabilidad	Migración del modelo LLM de Ollama local hacia OpenAI en nube	Cambio de configuración en .env	Los agentes continúan consumiendo el contrato unificado `/v1/generate` del LLM Gateway sin alterar una sola línea de código.	Esfuerzo de adaptación igual a cero horas en los agentes (solo variable de entorno).

---

11. RIESGOS, DEUDA TÉCNICA Y EVOLUCIÓN DEL SISTEMA

11.1. Matriz de Trazabilidad Cruzada Oficial (Anexo A SRS)

Restricción ToC	Descripción del Cuello de Botella	Requisitos Funcionales IEEE 830	Interfaces de Software	Cobertura en la Arquitectura de la PoC
R1	Bucle de edición de video y retrabajo interno ($IC=15/15$)	REQ-F11 a REQ-F20	REQ-IE03, REQ-IE07, REQ-IE10	**Diseño Estructural Completo** (Validación determinista en `validate_dossier` y trazabilidad).
R2	Ineficiencia en fase de ideación y briefing ($IC=14/15$)	REQ-F01 a REQ-F10	REQ-IE01, REQ-IE02, REQ-IE04	**100% Implementado y Validado** (Agente de Ideación, RAG y Google News RSS).
R3	Adaptación manual para publicación multicanal ($IC=12/15$)	REQ-F21 a REQ-F30	REQ-IE04, REQ-IE05, REQ-IE08	**100% Implementado y Validado** (Agente de Publicación, variantes y Postiz API).

11.2. Validación Formal con Stakeholders (Anexo B SRS)

Conforme al Anexo B de la Especificación de Requisitos de Software: - Se ejecutó un **Taller de Co-creación y Validación Técnica de 2 horas** con el equipo directivo y creativo de ITY SAS. - Se evaluaron los 49 requisitos bajo la metodología **MoSCoW**: 20 Must Have (40,8%), 18 Should Have (36,7%), 11 Could Have (22,5%) y 0 Won't Have. - El consenso formal inicial alcanzó el **93,9%**, superando el umbral institucional mínimo exigido del 90%. - Tras ajustes menores de redacción en los requisitos `REQ-F03`, `REQ-F16` y `REQ-NF07`, se obtuvo una **aprobación unánime del 100%** mediante acta formal de validación.

11.3. Síntesis de la Ruta de Generación Implementada en la PoC



Figura 11.1: Síntesis de la Ruta de Generación Implementada en la Prueba de Concepto.

La PoC implementa una ruta reproducible y determinista alrededor del modelo generativo: 1. Reúne el contexto de marca y mercado. 2. Inyecta restricciones técnicas específicas por plataforma. 3. Valida rígidamente la salida estructurada en JSON. 4. Persiste los resultados con enriquecimiento vectorial en pgvector. 5. Brinda compuertas de control humano antes de cualquier interacción con el mundo exterior.

11.4. Roadmap de Capacidades hacia el Escenario Productivo

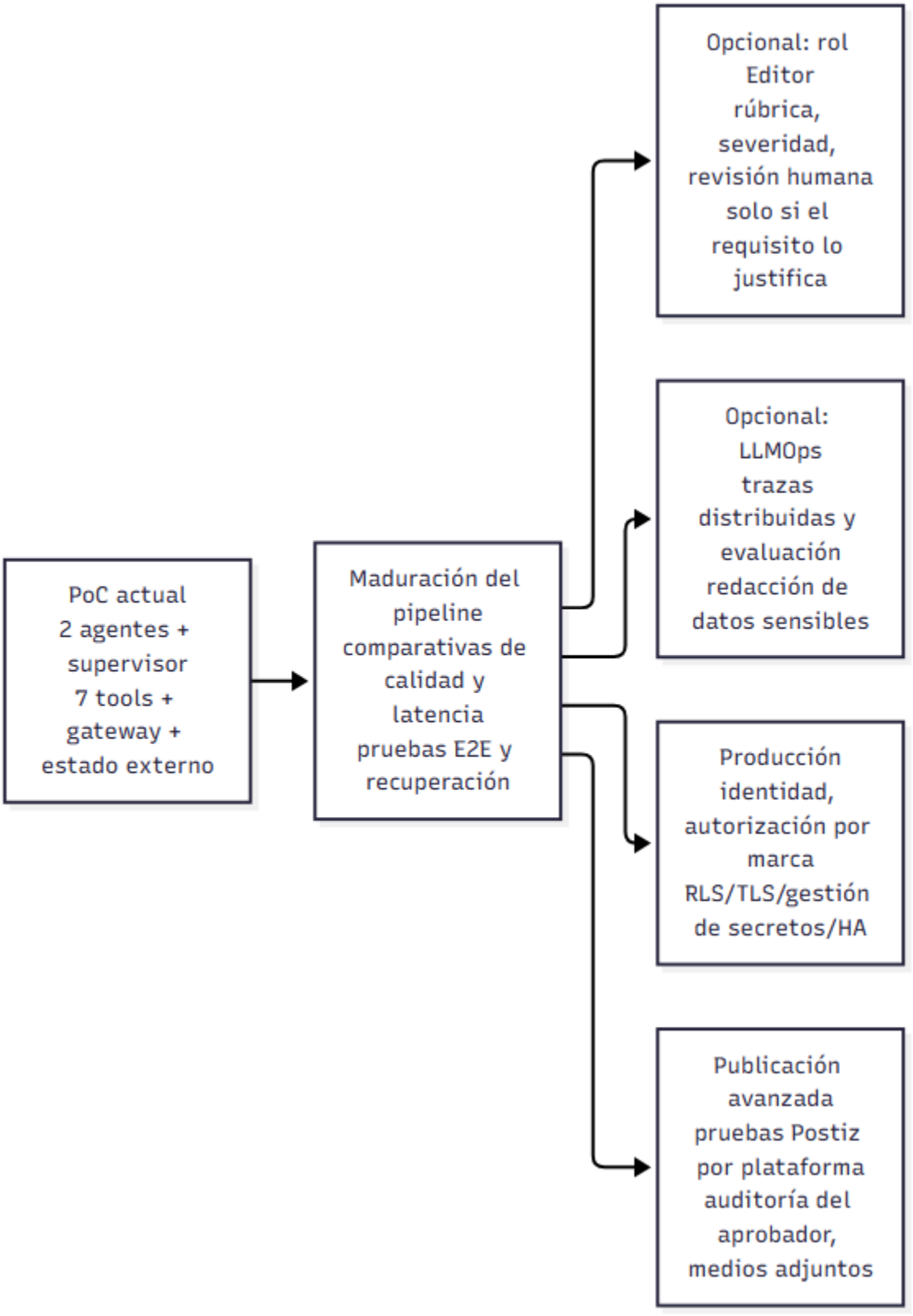


Figura 11.2: Roadmap de Evolución de Capacidades hacia la Arquitectura Productiva.

El plan de evolución tecnológica proyecta tres fases de maduración: - **Horizonte 1 (PoC Actual - Completada):** 14 contenedores Docker Compose, supervisor LangGraph, 2 agentes, 7 tools, PostgreSQL/pgvector, Redis, RabbitMQ y Postiz local. - **Horizonte 2 (Despliegue Empresarial en Servidor Dedicado):** Autenticación de usuarios vía Keycloak (OAuth2/OIDC), proxy inverso Traefik con TLS automatizado, ingesta de analítica de engagement en tiempo real e integración de HashiCorp Vault en producción. - **Horizonte 3 (Cluster Cloud Kubernetes):** Orquestación mediante Helm Charts, autoscaling horizontal de inferencia (HPA), réplicas distribuidas de PostgreSQL y almacenamiento en la nube sobre AWS S3 / Google Cloud Storage.

---

12. GLOSARIO DE TÉRMINOS DE INGENIERÍA

Término / Acrónimo	Definición Técnica en el Contexto del Sistema
ADR	*Architecture Decision Record:* Documento formal que captura una decisión de diseño arquitectónico relevante junto con su contexto, alternativas y consecuencias.
AOMS	*Agent-Oriented MicroServices:* Paradigma de arquitectura que fusiona los principios de autonomía y deliberación de los sistemas multiagente con la escalabilidad y desacoplamiento de los microservicios.
ASR	*Automatic Speech Recognition:* Tecnología de reconocimiento automático del habla empleada en REQ-F15 para generar subtítulos sincronizados en formato SRT.
ATAM	*Architecture Tradeoff Analysis Method:* Marco metodológico del SEI para evaluar arquitecturas de software analizando compromisos entre atributos de calidad.
AUML	*Agent Unified Modeling Language:* Extensión de la notación UML diseñada específicamente para modelar protocolos de comunicación, roles y concurrencia en sistemas multiagente.
BDI	*Belief-Desire-Intention:* Modelo cognitivo que conceptualiza a un agente inteligente a través de sus Creencias (estado del mundo), Deseos (objetivos) e Intenciones (planes de acción comprometidos).
BFF	*Backend for Frontend:* Patrón arquitectónico donde un servicio intermedio (`frontend-bff` en FastAPI) adapta, agrega y expone datos específicamente optimizados para la SPA web.
Checkpoint	Instantánea serializada del estado interno de un grafo deliberativo en Redis bajo un `thread_id` único, permitiendo pausar y reanudar flujos de forma idempotente.
Dossier Creativo	Estructura JSON validada que contiene el guion técnico, desglose de escenas, gancho conversacional, llamado a la acción, copies adaptados y hashtags de una pieza publicitaria.
HITL	*Human-in-the-Loop:* Patrón de gobernanza que exige la validación y autorización explícita de un operador humano antes de que el sistema ejecute acciones que alteren el entorno exterior.

Término / Acrónimo	Definición Técnica en el Contexto del Sistema
HPA	*Horizontal Pod Autoscaler:* Mecanismo de Kubernetes para escalar automáticamente el número de réplicas de pods en función del consumo de CPU o memoria (REQ-NF05).
IVFFlat	*Inverted File Flat:* Índice de vecinos aproximados utilizado en `pgvector` para acelerar drásticamente la búsqueda semántica de vectores de alta dimensión.
LangGraph	Marco de ejecución de grafos deliberativos que permite modelar agentes autónomos con soporte nativo de ciclos, bifurcaciones condicionales, interrupciones y checkpoints externos.
LLM Gateway	Microservicio fachada que administra de manera centralizada el enrutamiento, balanceo, reintentos con backoff exponencial, fallback y caché de inferencias hacia los modelos de lenguaje.
LUFS	*Loudness Units Full Scale:* Estándar internacional (ITU-R BS.1770) para medir el volumen sonoro percibido en transmisiones y video digital (requisito -16 a -12 LUFS en REQ-F11).
pgvector	Extensión de código abierto para PostgreSQL que agrega soporte nativo para almacenamiento de vectores y búsqueda por similitud de coseno, producto interno y distancia euclidiana.
Postiz	Plataforma de código abierto autohospedable para la programación, gestión y publicación de contenidos en múltiples redes sociales simultáneas.
RAG	*Retrieval-Augmented Generation:* Técnica que complementa el prompt de un modelo generativo con fragmentos semánticamente relevantes recuperados desde una base vectorial externa.
RedisSaver	Componente oficial de LangGraph que implementa la persistencia de checkpoints de ejecución en instancias de base de datos Redis.
Rough Cut	Corte de edición preliminar generado algorítmicamente a partir de una plantilla base para reducir los tiempos de trabajo manual del editor (REQ-F13).
STRIDE	Metodología de modelado de amenazas informáticas desarrollada por Microsoft para categorizar vectores de ataque (*Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege*).
TaskStore	Componente en memoria estructurado sobre Redis para gestionar el ciclo de vida, progreso agregado y estados de las tareas y publicaciones del sistema.
Tool Discovery	Mecanismo dinámico que permite a los agentes localizar e invocar herramientas inspeccionando contratos expuestos en la ruta normalizada `/.well-known/tool-manifest`.

13. REFERENCIAS BIBLIOGRÁFICAS

1. **IEEE Computer Society.** (1998). *IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications*. Institute of Electrical and Electronics Engineers.
2. **ISO/IEC/IEEE.** (2022). *ISO/IEC/IEEE 42010:2022 Systems and software engineering -- Architecture description*. International Organization for Standardization.
3. **Brown, S.** (2018-2024). *The C4 model for visualising software architecture*. Recuperado de: https://c4model.com/
4. **Wooldridge, M., Jennings, N. R., & Kinny, D.** (2000). The Gaia methodology for agent-oriented analysis and design. *Autonomous Agents and Multi-Agent Systems*, 3(3), 285-312.
5. **Odell, J., Parunak, H. V. D., & Bauer, B.** (2000). Extending UML for agents. In *Proceedings of the Agent-Oriented Information Systems Workshop (AOIS)*, pp. 3-17.
6. **Rao, A. S., & Georgeff, M. P.** (1995). BDI agents: From theory to practice. In *Proceedings of the First International Conference on Multi-Agent Systems (ICMAS)*, pp. 312-319.
7. **Kazman, R., Klein, M., & Clements, P.** (2000). *ATAM: Method for architecture evaluation*. Technical Report CMU/SEI-2000-TR-004, Software Engineering Institute, Carnegie Mellon University.
8. **Goldratt, E. M., & Cox, J.** (2004). *The Goal: A Process of Ongoing Improvement* (3rd ed.). North River Press.
9. **Rother, M., & Shook, J.** (2003). *Learning to See: Value Stream Mapping to Add Value and Eliminate MUDA*. Lean Enterprise Institute.
10. **OpenAPI Initiative.** (2021). *OpenAPI Specification v3.0.3*. Linux Foundation. Recuperado de: https://spec.openapis.org/oas/v3.0.3
11. **LangChain Inc.** (2024-2025). *LangGraph: Build resilient multi-agent systems with human-in-the-loop and external persistence*. Recuperado de: https://langchain-ai.github.io/langgraph/
12. **VMware Inc.** (2024). *RabbitMQ Reliability Guide and Highly Available Queues*. Recuperado de: https://www.rabbitmq.com/docs/reliability
13. **Microsoft Corporation.** (2024). *The STRIDE Threat Model*. Microsoft Security Development Lifecycle.
14. **Congreso de la República de Colombia.** (2012). *Ley Estatutaria 1581 de 2012: Régimen General de Protección de Datos Personales*. Diario Oficial No. 48.587.

FIN DEL DOCUMENTO DE ARQUITECTURA DE SOFTWARE (SAD) **Universidad Católica de Colombia • Facultad de Ingeniería** *Bogotá D.C., Colombia -- 2026*